ASCEND 2026 · C-DAC MUMBAI

Study Notes — October Sessions

Applied AI/ML Engineering · rebuilt from the sessions of 6–23 October 2026. Every code cell was run; its output is printed beneath it.

1. 6–8 October — Relationships in Data: Correlation, Regression & Hypothesis Tests
2. 14–15 October — NumPy & pandas Foundations: Arrays, Conditions, Selecting Rows
3. 15–17 October — Combining, Cleaning & Reshaping Tables: merge, groupby, pivot
4. 17–21 October — Data Visualisation: Which Chart for Which Data
5. 21 October — Describing Data: Centre, Spread & Shape
6. 22 October — Probability & Bayes' Theorem
7. 23 October — Probability Distributions with scipy

6–8 October — Relationships in Data: Correlation, Regression & Hypothesis Tests

Correlation · Simple regression · Two-sample test · Dummy variables · Logistic regression · Chi-square

These notes rebuild what Sudeep Bhargava taught in the three sessions, in the order he taught it, using the same variable names and the same CDAC_DataBook.xlsx sheets (faithful, salaries, diabetes). Every runnable cell below uses SIMULATED data with a fixed seed, because the Excel file is not available inside the browser. Your numbers in class (for example p = 0.0054 for the gender test) will therefore differ slightly from the numbers printed here. The code is identical, so paste it into your notebook and swap the simulated lines for pd.read_excel(...).

How to read the grades:

- 🟢 Start here — the idea and the one line of code
- 🔵 Core — what the exam and the assignments ask
- 🟠 Go deeper — why it works, common traps
- 🔴 Stretch — optional, for the curious

🟢 1 · The one question to ask first

Before choosing any method, name your variables. Everything taught in these three sessions follows from two labels on each column:

- ROLE — is it the response (the thing you want to explain or predict, also called y or the output) or a predictor (the thing you use to explain it, also called x or the input)?
- TYPE — is it continuous (numbers on a scale: salary, waiting time, glucose) or categorical (labels: gender, rank, diet type)? A categorical variable with exactly two labels is binary (Outcome = 0/1).

Examples from class:

- salary — response, continuous
- yrs_service — predictor 1, continuous
- gender — predictor 2, categorical
- Outcome (diabetes yes/no) — response, binary
- Glucose, BloodPressure, Age — continuous predictors; DietType — categorical predictor

Which tool fits which pair:

```
Predictor      Response       Look at it with        Test / model
continuous     continuous     scatter plot           Pearson correlation, linear regression
categorical    continuous     box plot               2-sample test (2 groups), regression with
dummy variables
categorical    categorical    crosstab, bar chart    chi-square test of independence
mixed          binary         —                      logistic regression
```

🟢 2 · Correlation — how tightly do two numbers move together?

The Pearson correlation coefficient r measures the strength and direction of a STRAIGHT-LINE relationship between two continuous variables.

- r is always between −1 and +1
- r close to +1: when x goes up, y goes up, almost on a line
- r close to −1: when x goes up, y goes down
- r close to 0: no straight-line relationship (there could still be a curved one)
- Correlation is NOT causation — it only says the two move together

Faculty code (the faithful sheet: waiting = minutes between eruptions, eruptions = length of an eruption):

```
df = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='faithful')
x = df.waiting
y = df.eruptions
from scipy.stats import pearsonr
pearsonr(x, y)   # value of the correlation coefficient
```

pearsonr returns two numbers: r and a p-value. The p-value belongs to the hypothesis test:

- Ho (null): the true correlation is 0, i.e. y does not depend on x
- If p < 0.05, REJECT Ho — there is evidence of a relationship
- If p ≥ 0.05, fail to reject Ho — no evidence

In class, for salary vs yrs_service the p-value printed as 0, which is below 0.05, so the conclusion written on the board was: "salary depends on yrs_service".

▶ Try it — Correlation on simulated faithful-style data

```
import numpy as np
from scipy.stats import pearsonr

rng = np.random.default_rng(42)
waiting = rng.normal(70, 13, 272).round()
eruptions = -1.87 + 0.0756 * waiting + rng.normal(0, 0.4, 272)

r, p = pearsonr(waiting, eruptions)
print("r       = %.3f" % r)
print("p-value = %.2e" % p)
if p < 0.05:
    print("p < 0.05  ->  reject Ho: eruptions depend on waiting time")
else:
    print("p >= 0.05 ->  cannot reject Ho")
```

OUTPUT

```
r       = 0.908
p-value = 5.99e-104
p < 0.05  ->  reject Ho: eruptions depend on waiting time
```

🔵 3 · Simple linear regression — draw the best line

Regression turns "they move together" into a usable equation:

```
eruptions = intercept + coef × waiting
```

- coef (slope): how much y changes when x increases by 1 unit
- intercept: the value of y when x = 0
- R^2 (from regressor.score): the share of the variation in y explained by x. Between 0 and 1; closer to 1 means a better fit. Read it the same way as r: closer to 1 (or −1 for r) is stronger.

Faculty code on 6 October:

```
import sklearn
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
x = df.waiting
y = df.eruptions
x = np.array(x).reshape(-1, 1)
regressor.fit(x, y)
print(regressor.coef_)
print(regressor.intercept_)
```

🟠 Why reshape(-1, 1)? scikit-learn expects x as a TABLE (rows = observations, columns = features), even if there is only one feature. A plain column of numbers is a 1-D array, so we turn it into n rows × 1 column. The −1 means "Python, work out n yourself" (see the NumPy note of 14 October).

▶ Try it — Fit a line, read the coefficients, predict

```
import numpy as np
from sklearn.linear_model import LinearRegression

rng = np.random.default_rng(42)
waiting = rng.normal(70, 13, 272).round()
eruptions = -1.87 + 0.0756 * waiting + rng.normal(0, 0.4, 272)

x = np.array(waiting).reshape(-1, 1)
y = eruptions
regressor = LinearRegression()
regressor.fit(x, y)

print("coef      :", regressor.coef_)
print("intercept :", regressor.intercept_)
print("R-squared : %.3f" % regressor.score(x, y))
print("predicted eruption after a 80-minute wait: %.2f" % regressor.predict([[80]])[0])
```

OUTPUT

```
coef      : [0.07387926]
intercept : -1.7468523227915531
R-squared : 0.824
predicted eruption after a 80-minute wait: 4.16
```

🔵 4 · Does a categorical variable change a continuous one? (2-sample test)

Question from the salaries sheet: do male and female staff earn the same on average?

Step 1 — look first. A box plot of salary split by gender shows the two groups side by side.

Step 2 — split the response into the two groups:

```
mydf = df[['yrs_service', 'gender', 'salary']]
x1 = mydf.salary.iloc[np.where(mydf.gender == 'Male')]     # salaries of the male rows
x2 = mydf.salary.iloc[np.where(mydf.gender == 'Female')]   # salaries of the female rows
plt.boxplot([x1, x2])
```

Step 3 — test:

- Ho: average salary of males = average salary of females
- Ha: the averages are different

```
import statsmodels
from statsmodels import stats
from statsmodels.stats import weightstats as ssw
ssw.ztest(x1, x2)
# p-value = 0.0054 < 0.05  ->  reject Ho
# average salary of males is not equal to average salary of females
```

Rule for the p-value: small p (below the 0.05 cut-off) means the data would be very surprising if Ho were true, so we reject Ho.

🟠 z-test or t-test? With large groups (30 or more in each) the z-test and the 2-sample t-test give nearly identical answers. The board called it a "2 sample t test"; the code used was ztest. Both ideas are tested in the cell below. The statsmodels line above is shown for reference — the cell uses scipy so that it runs in your browser.

▶ Try it — Two-sample comparison on simulated salaries

```
import numpy as np
from scipy.stats import norm, ttest_ind

rng = np.random.default_rng(7)
x1 = rng.normal(115000, 30000, 300)   # "male" salaries
x2 = rng.normal(101000, 28000, 60)    # "female" salaries

# z statistic by hand (what ztest does)
se = np.sqrt(x1.var(ddof=1)/len(x1) + x2.var(ddof=1)/len(x2))
z = (x1.mean() - x2.mean()) / se
p_z = 2 * (1 - norm.cdf(abs(z)))

t_stat, p_t = ttest_ind(x1, x2, equal_var=False)
print("mean male   = %.0f" % x1.mean())
print("mean female = %.0f" % x2.mean())
print("z-test  p = %.4f" % p_z)
print("t-test  p = %.4f" % p_t)
print("Reject Ho (means equal)?", p_z < 0.05)
```

OUTPUT

```
mean male   = 111044
mean female = 102168
z-test  p = 0.0109
t-test  p = 0.0126
Reject Ho (means equal)? True
```

🔵 5 · Putting a category into a regression: dummy variables

A regression equation only understands numbers. A column of words like Male / Female must be converted first. The trick is a DUMMY (0/1) column:

```
g_dummy = pd.get_dummies(mydf.gender, drop_first=True).astype(int)
mydf = mydf.drop('gender', axis=1)
mydf = pd.concat([mydf, g_dummy], axis=1)
x = mydf.drop('salary', axis=1)     # inputs
y = mydf.salary                     # output
from sklearn.linear_model import LinearRegression
reg1 = LinearRegression()
reg1.fit(x, y)
reg1.coef_
reg1.intercept_
```

What happens: gender has two labels, so we keep ONE dummy column (Male = 1, Female = 0). Female becomes the REFERENCE LEVEL — the label that is left out and treated as the baseline. Alphabetical order decides the reference (Female comes before Male). drop_first=True does the dropping.

How to read the output (the sentences from the board):

- coefficient of yrs_service = 747.61 → when yrs_service goes up by 1 unit, salary goes up by 747.61, holding gender fixed
- coefficient of the Male dummy = 9071.8 → for the SAME years of service, choosing a male candidate over a female means paying 9071.8 more
- intercept = 92356 → the salary of a FEMALE (reference level) with yrs_service = 0

The general rule for the intercept: it is the predicted response when every continuous predictor is 0 AND every categorical predictor sits at its reference level.

🟠 Why drop one column? If you kept both Female and Male dummies, one is always 1 − the other, so the model cannot tell them apart (the "dummy variable trap"). A variable with k labels needs k − 1 dummy columns.

▶ Try it — Dummy-variable regression (simulated with known answers)

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

rng = np.random.default_rng(1)
n = 800
yrs = rng.integers(0, 40, n)
gender = np.where(rng.random(n) < 0.8, "Male", "Female")
salary = 92356 + 747.6*yrs + 9071.8*(gender == "Male") + rng.normal(0, 15000, n)
mydf = pd.DataFrame({"yrs_service": yrs, "gender": gender, "salary": salary})

g_dummy = pd.get_dummies(mydf.gender, drop_first=True).astype(int)
mydf = pd.concat([mydf.drop("gender", axis=1), g_dummy], axis=1)
print("columns now:", list(mydf.columns))

x = mydf.drop("salary", axis=1)
y = mydf.salary
reg1 = LinearRegression().fit(x, y)
for name, c in zip(x.columns, reg1.coef_):
    print("%-12s coefficient = %9.1f" % (name, c))
```

```
print("intercept (female, 0 yrs) = %.0f" % reg1.intercept_)
print("true values used: 747.6, 9071.8, 92356")
```

OUTPUT

```
columns now: ['yrs_service', 'salary', 'Male']
yrs_service  coefficient =     766.2
Male         coefficient =    9706.3
intercept (female, 0 yrs) = 92133
true values used: 747.6, 9071.8, 92356
```

🔵 6 · A category with more than two labels (rank)

On 8 October the same salaries sheet was used with rank, which has three labels: AsstProf, AssocProf, Prof.

```
df = df[['rank', 'yrs_service', 'salary']]
rank_dummy = pd.get_dummies(df['rank'], drop_first=True).astype(int)
df = df.drop('rank', axis=1)
df = pd.concat([df, rank_dummy], axis=1)
x = df.drop('salary', axis=1)    # input columns
y = df.salary                    # output column
regr = LinearRegression()
regr.fit(x, y)
regr.coef_
```

Three labels → two dummy columns (AsstProf, Prof). The reference level is AssocProf (first alphabetically). The coefficients on the board and what they mean:

```
Coefficient   Belongs to     Reading
−158          yrs_service    each extra year, rank unchanged: salary falls by 158
−14615        AsstProf       AssocProf → AsstProf, same years: salary falls by 14615
+34613        Prof           AssocProf → Prof, same years: salary rises by 34613
95767         intercept      yrs_service = 0 and rank = AssocProf: salary is 95767
```

Always say "compared with the reference level" — a dummy coefficient is a DIFFERENCE from the baseline, never an absolute salary.

▶ Try it — Three-level categorical predictor

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

rng = np.random.default_rng(3)
n = 800
rank = rng.choice(["AsstProf", "AssocProf", "Prof"], n, p=[0.2, 0.2, 0.6])
yrs = rng.integers(0, 40, n)
salary = (95767 - 158*yrs - 14615*(rank == "AsstProf")
          + 34613*(rank == "Prof") + rng.normal(0, 6000, n))
df = pd.DataFrame({"rank": rank, "yrs_service": yrs, "salary": salary})

rank_dummy = pd.get_dummies(df["rank"], drop_first=True).astype(int)
df = pd.concat([df.drop("rank", axis=1), rank_dummy], axis=1)
x = df.drop("salary", axis=1)
y = df.salary
regr = LinearRegression().fit(x, y)
```

```
print("reference level is the one MISSING from:", list(x.columns))
for name, c in zip(x.columns, regr.coef_):
    print("%-12s %9.0f" % (name, c))
print("intercept    %9.0f" % regr.intercept_)
```

OUTPUT

```
reference level is the one MISSING from: ['yrs_service', 'AsstProf', 'Prof']
yrs_service       -159
AsstProf        -14270
Prof             34433
intercept        95971
```

🔵 7 · Binary response → logistic regression

When the response is yes/no (diabetes: Outcome = 1 or 0), a straight line is a bad fit because probabilities must stay between 0 and 1. Logistic regression squeezes the line into an S-curve and predicts the PROBABILITY of Outcome = 1.

Setup from the diabetes sheet:

```
df = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='diabetes')
df = df[['Glucose', 'BloodPressure', 'Age', 'DietType', 'Outcome']]
# 'Glucose', 'BloodPressure', 'Age'  - continuous predictors
# DietType                           - categorical predictor
# Outcome                            - binary response
diet_dummy = pd.get_dummies(df['DietType'], prefix='Diet', drop_first=True).astype(int)
df = df.drop('DietType', axis=1)
df = pd.concat([df, diet_dummy], axis=1)
```

```
from sklearn.linear_model import LogisticRegression
log_mod = LogisticRegression()
x = df.drop('Outcome', axis=1)
y = df.Outcome
log_mod.fit(x, y)
log_mod.coef_
```

Reading the coefficients (from the board):

- Glucose coefficient is POSITIVE → raising glucose increases the probability of diabetes
- Diet_2 coefficient is NEGATIVE → changing diet from Diet1 (reference) to Diet2 decreases the probability of diabetes

Same dummy-variable logic as in section 5. The sign tells the direction; the size is on the log-odds scale.

🟠 The magnitude is not "probability points". exp(coefficient) is the odds multiplier: exp(0.05) ≈ 1.05, so each extra unit of glucose multiplies the odds of diabetes by about 1.05, holding the other columns fixed.

▶ Try it — Logistic regression on simulated diabetes-style data

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(11)
n = 3000
```

```
df = pd.DataFrame({
    "Glucose": rng.normal(120, 30, n).round(),
    "BloodPressure": rng.normal(70, 12, n).round(),
    "Age": rng.integers(21, 70, n),
    "DietType": rng.choice([1, 2, 3], n),
})
logit = -9 + 0.06*df.Glucose + 0.015*df.Age - 0.8*(df.DietType == 2)
df["Outcome"] = (rng.random(n) < 1/(1 + np.exp(-logit))).astype(int)

diet_dummy = pd.get_dummies(df["DietType"], prefix="Diet", drop_first=True).astype(int)
df = pd.concat([df.drop("DietType", axis=1), diet_dummy], axis=1)
x = df.drop("Outcome", axis=1)
y = df.Outcome

log_mod = LogisticRegression(max_iter=2000).fit(x, y)
for name, c in zip(x.columns, log_mod.coef_[0]):
    print("%-14s %7.3f   odds multiplier %.3f" % (name, c, np.exp(c)))
one = pd.DataFrame([[150, 70, 50, 0, 0]], columns=x.columns)
print("P(diabetes) for glucose 150, age 50, Diet1 = %.2f" % log_mod.predict_proba(one)[0][1])
```

OUTPUT

```
Glucose          0.059   odds multiplier 1.061
BloodPressure    0.003   odds multiplier 1.003
Age              0.017   odds multiplier 1.018
Diet_2          -0.919   odds multiplier 0.399
Diet_3          -0.086   odds multiplier 0.918
P(diabetes) for glucose 150, age 50, Diet1 = 0.70
```

🔵 8 · Two categories against each other: chi-square test

Is Outcome related to DietType? Both are categorical, so no averages exist — we compare COUNTS.

```
newdf = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='diabetes')
diet_counts = pd.DataFrame(newdf.DietType.value_counts())
diet_counts.plot(kind='bar')               # how many people on each diet
abc = pd.crosstab(newdf.Outcome, newdf.DietType)
abc.plot(kind='bar')                       # Outcome counts inside each diet
from scipy.stats import chi2_contingency
chi2_contingency(abc)
```

- Ho: Outcome and DietType are independent (diet makes no difference)
- Ha: they are associated
- p < 0.05 → reject Ho

crosstab builds the table of counts; chi2_contingency compares it with the table you would see if the two were unrelated. It returns four things: the chi-square statistic, the p-value, the degrees of freedom and the expected counts.

Class also split the diabetes data by age (Age ≤ 35 vs Age > 35) and drew a bar chart of Outcome for each group — a quick visual way to see whether age changes the outcome.

▶ Try it — Chi-square test of independence

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency
```

```
rng = np.random.default_rng(5)
n = 600
diet = rng.choice(["Diet1", "Diet2", "Diet3"], n)
p_sick = np.where(diet == "Diet2", 0.20, 0.38)
outcome = (rng.random(n) < p_sick).astype(int)
abc = pd.crosstab(outcome, diet)
print(abc, "\n")

chi2, p, dof, expected = chi2_contingency(abc)
print("chi-square = %.2f   dof = %d   p-value = %.5f" % (chi2, dof, p))
print("Reject Ho (independent)?", p < 0.05)
```

OUTPUT

```
col_0  Diet1  Diet2  Diet3
row_0
0        133    147    117
1         74     49     80

chi-square = 11.21   dof = 2   p-value = 0.00368
Reject Ho (independent)? True
```

🟠 9 · Mistakes seen in class — avoid them

- Mixing data frames in one condition: df.salary.iloc[np.where(df.gender=='Female')] — make sure the column you test comes from the same frame as the column you pull values from.
- Forgetting .astype(int) after get_dummies: newer pandas returns True/False. Convert to 0/1 before fitting.
- Forgetting to remove the original text column (drop('gender', axis=1)) before fitting: the model cannot handle words.
- Reading a dummy coefficient as an absolute value. It is always "compared with the reference level".
- Reading the intercept literally when 0 is not realistic (0 years of service may be outside the data).
- Treating a small p-value as proof of cause. It only shows the pattern is unlikely to be luck.

🔵 10 · Practice questions

Q1. Name the response and predictor types, and the test, for: (a) height vs weight, (b) department vs salary (3 departments), (c) smoker (yes/no) vs lung disease (yes/no).

Q2. A regression gives salary = 80000 + 900 × yrs + 12000 × Male (reference: Female). What is the predicted salary of a woman with 10 years of service? A man with 10 years?

Q3. A gender variable has labels Female / Male / Other. How many dummy columns are needed and which label is the reference?

Q4. pearsonr gives r = −0.82, p = 0.00001. Write the conclusion in one sentence.

Q5. A logistic regression has coefficient +0.04 for Age. Does older age raise or lower the probability of the outcome? By what factor does one extra year multiply the odds (approximately)?

Q6. chi2_contingency returns p = 0.31 for Outcome vs DietType. What do you conclude about Ho?

Answers

A1. (a) both continuous → scatter plot, Pearson correlation or simple regression. (b) categorical predictor, continuous response → box plots; regression with 2 dummy columns (or ANOVA, later). (c) both categorical → crosstab and chi-square test.

A2. Woman: 80000 + 9000 = 89000. Man: 89000 + 12000 = 101000.

A3. Three labels → two dummy columns. Reference = first alphabetically = Female.

A4. Strong negative linear relationship; p < 0.05, so reject Ho — as x increases, y decreases.

A5. Raises it. exp(0.04) ≈ 1.04, so odds rise by about 4% per year.

A6. p > 0.05 → fail to reject Ho: no evidence that Outcome depends on DietType.

🟢 11 · One-page cheat sheet

```
pearsonr(x, y)                          r and p for two continuous columns
LinearRegression().fit(X, y)            .coef_  .intercept_  .score(X, y)
np.array(x).reshape(-1, 1)              one feature column for sklearn
ssw.ztest(x1, x2)                       compare two group means
pd.get_dummies(col, drop_first=True)    k labels -> k-1 dummy columns
.astype(int)                            True/False -> 1/0
LogisticRegression().fit(X, y)          binary response; sign of coef = direction
pd.crosstab(a, b)                       table of counts
chi2_contingency(table)                 statistic, p, dof, expected
```

- p < 0.05 → reject Ho.
- Dummy coefficient = difference from the reference level.
- Intercept = response when numeric predictors are 0 and categories are at the reference level.

🔴 12 · Also on 8 October — reading list

The afternoon of 8 October moved on to three new algorithms. The reading links shared in the class chat were:

- Support Vector Machine: https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/
- Apriori (association rules): https://www.geeksforgeeks.org/machine-learning/apriori-algorithm/
- Principal Component Analysis: https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/

14–15 October — NumPy & pandas Foundations: Arrays, Conditions, Selecting Rows

Arrays · Indexing and slicing · reshape · Conditions with np.where · Reading Excel · loc and iloc · drop · concat

Sudeep Bhargava started the Python data tools from the ground up. These notes follow the board and the code he pasted in the class chat, with the same names (a1, ind, df, mydf, MWH-2 ...). All runnable cells use small arrays you can check by hand or simulated ERP-style data with a fixed seed, so every result below can be reproduced in your browser.

How to read the grades: 🟢 start here · 🔵 core · 🟠 go deeper · 🔴 stretch

🟢 1 · The two libraries and their data structures

Every notebook of this course starts with the same imports:

```
import numpy as np
import pandas as pd
import matplotlib
from matplotlib import pyplot as plt
```

What Python already gives you (built-in containers): list, tuple, dictionary, set.

What the two libraries add:

- NumPy → ndarray. The "nd" means n-dimensional: a block of numbers of the same type in 1, 2 or more dimensions. Fast arithmetic on all elements at once.
- pandas → Series (one labelled column) and DataFrame (a table of columns, like an Excel sheet).

Rule of thumb: raw numbers and maths → NumPy; labelled tables from Excel/CSV → pandas. A DataFrame column is built on top of NumPy, which is why np.where works on df columns later in these notes.

🟢 2 · Creating and indexing an array

```
a1 = np.array([3, 6, 5, 7, 8, 9, 1, 2])   # creating an array
type(a1)     # to know the type of data structure
len(a1)      # number of elements
a1[3]        # extract the fourth element
```

Python counts from 0. The first element is a1[0], the fourth is a1[3].

Picking several positions at once — put the positions in a list inside the brackets:

```
a1[[2, 5, 6]]   # extracts the elements at index 2, 5 and 6
```

Slicing — start:stop, where the stop index is NOT included:

```
a1[2:5]    # index 2, 3, 4 (= 5 - 1)
a1[2:]     # from index 2 up to the end
```

```
a1[:4]     # from the start up to index 3 (= 4 − 1)
```

▶ Try it — Indexing and slicing, check against the numbers above

```
import numpy as np
a1 = np.array([3, 6, 5, 7, 8, 9, 1, 2])
print("type          :", type(a1))
print("len           :", len(a1))
print("a1[3]         :", a1[3])
print("a1[[2,5,6]]   :", a1[[2, 5, 6]])
print("a1[2:5]       :", a1[2:5])
print("a1[2:]        :", a1[2:])
print("a1[:4]        :", a1[:4])
```

OUTPUT

```
type          : <class 'numpy.ndarray'>
len           : 8
a1[3]         : 7
a1[[2,5,6]]   : [5 9 1]
a1[2:5]       : [5 7 8]
a1[2:]        : [5 7 8 9 1 2]
a1[:4]        : [3 6 5 7]
```

🔵 3 · Two-dimensional arrays and reshape

```
a2 = np.array([[2, 4, 5, 6], [1, 6, 5, 7]])   # 2 rows × 4 columns
```

```
a3 = np.array([4, 3, 5, 6, 8, 7, 9, 2])
a4 = a3.reshape(4, 2)     # 4 rows and 2 columns
a5 = a3.reshape(-1, 1)    # n rows and 1 column
a6 = a3.reshape(2, 4)     # 2 rows and 4 columns
```

- reshape never changes the values or their order — only how they are cut into rows and columns.
- The product of rows × columns must equal the number of elements (8 here).
- −1 means "let Python work out this number". With reshape(-1, 1) the −1 becomes 8, the number of elements of a3. You will use reshape(-1, 1) every time you give a single feature column to scikit-learn.

▶ Try it — reshape

```
import numpy as np
a3 = np.array([4, 3, 5, 6, 8, 7, 9, 2])
print("shape of a3 :", a3.shape)
print("reshape(4,2):\n", a3.reshape(4, 2))
print("reshape(2,4):\n", a3.reshape(2, 4))
print("reshape(-1,1) gives shape", a3.reshape(-1, 1).shape)
a2 = np.array([[2, 4, 5, 6], [1, 6, 5, 7]])
print("a2 shape    :", a2.shape, "(2 rows, 4 columns)")
```

OUTPUT

```
shape of a3 : (8,)
reshape(4,2):
 [[4 3]
```

```
 [5 6]
 [8 7]
 [9 2]]
reshape(2,4):
 [[4 3 5 6]
 [8 7 9 2]]
reshape(-1,1) gives shape (8, 1)
a2 shape    : (2, 4) (2 rows, 4 columns)
```

🔵 4 · Conditions and np.where — the heart of these two days

A comparison on an array is applied to EVERY element and returns an array of True / False:

```
a1 = np.array([3, 6, 5, 7, 8, 9, 1, 2])
a1 <= 4          # True where the condition is met, False where it is not
ind = np.where(a1 <= 4)    # the INDICES where the condition is met
```

np.where returns a tuple. Index 0 of the tuple holds the array of positions, which is why you will always see ind[0].

Three questions, three answers (written on the board as Quest1–3):

```
Quest1: what are the indices of the values which are <= 4
Quest2: how many values are <= 4
Quest3: what are the values which are <= 4
```

```
ind = np.where(a1 <= 4)
ind[0]             # Q1: the indices
len(ind[0])        # Q2: how many
(a1 <= 4).sum()    # Q2 again: True counts as 1, so sum = count
a1[ind[0]]         # Q3: the values at those indices
a1[a1 <= 4]        # Q3 again: directly with the True/False mask
```

▶ Try it — The three questions

```
import numpy as np
a1 = np.array([3, 6, 5, 7, 8, 9, 1, 2])
print("mask      :", a1 <= 4)
ind = np.where(a1 <= 4)
print("Q1 indices:", ind[0])
print("Q2 count  :", len(ind[0]), "  or  ", (a1 <= 4).sum())
print("Q3 values :", a1[ind[0]], "  or  ", a1[a1 <= 4])
```

OUTPUT

```
mask      : [ True False False False False False  True  True]
Q1 indices: [0 6 7]
Q2 count  : 3   or   3
Q3 values : [3 1 2]   or   [3 1 2]
```

🔵 5 · More than one condition: AND, OR

Logical AND first, with plain True/False values (15 October):

```
p1 = True
p2 = False
p3 = True
p1 & p2    # AND  -> False (both must be True)
p1 & p3    # AND  -> True
```

Rule for the combined condition: write each condition in its OWN round brackets and join them with & (AND) or | (OR). The brackets are not optional.

```
a1 = np.array([3, 6, 5, 7, 9, 8, 7, 5, 4, 6, 5, 7, 10, 3])
# a1 >= 4 and a1 <= 7
ind = np.where((a1 >= 4) & (a1 <= 7))
# (array([ 1,  2,  3,  6,  7,  8,  9, 10, 11]),)
len(ind[0])                        # number of values meeting BOTH criteria
((a1 >= 4) & (a1 <= 7)).sum()      # same count, without np.where
a1[ind[0]]                         # values that meet both criteria
a1[(a1 >= 4) & (a1 <= 7)]          # same values, with the mask
```

🟠 Why not the words "and" / "or"? Python's and/or work on single True/False values. Array conditions need the element-by-element operators & and |.

▶ Try it — AND, OR and the bracket rule

```
import numpy as np
a1 = np.array([3, 6, 5, 7, 9, 8, 7, 5, 4, 6, 5, 7, 10, 3])
ind = np.where((a1 >= 4) & (a1 <= 7))
print("indices          :", ind[0])
print("count            :", len(ind[0]))
print("values           :", a1[ind[0]])
print("OR example (<4 or >9):", a1[(a1 < 4) | (a1 > 9)])
try:
    np.where(a1 >= 4 & a1 <= 7)
except Exception as e:
    print("without brackets ->", type(e).__name__)
```

OUTPUT

```
indices          : [ 1  2  3  6  7  8  9 10 11]
count            : 9
values           : [6 5 7 7 5 4 6 5 7]
OR example (<4 or >9): [ 3 10  3]
without brackets -> ValueError
```

🟢 6 · Where is my file? The working directory

pd.read_excel('CDAC_DataBook.xlsx') only works if Python is "standing" in the folder that holds the file.

```
import os
os.getcwd()                                  # where am I now?
os.chdir(r'C:\\Users\\Sudeep Bhargava\\Downloads')   # move to the folder with the file
```

Where students were working: VS Code (set the folder or use chdir); Google Colab (the working directory is /content; upload the file with the left-hand Files panel). Same code, different location.

🔵 7 · From Excel to DataFrame and first look

```
df = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='ERPData')
df.shape      # (rows, columns) — first number is rows, second is columns
df.head()     # first five rows
df.tail()     # last five rows
df.columns    # names of all columns
x1 = df.Quantity    # one column, as a Series
```

The ERPData sheet is a log of consignments with a MaterialID, a Location (warehouses such as MWH-1, MWH-2 …) and a Quantity.

Find the location of the heaviest consignment:

```
ind = np.where(df.Quantity == max(df.Quantity))
df.Location[ind[0]]      # location of the consignment with highest weight
df.MaterialID[ind[0]]    # MaterialID of that consignment
```

This is the same np.where pattern as on a1, only the array is now a DataFrame column.

▶ Try it — A tiny ERP table: shape, head, heaviest consignment

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(21)
n = 40
df = pd.DataFrame({
    "MaterialID": rng.choice(["M101", "M102", "M103", "M104"], n),
    "Location": rng.choice(["MWH-1", "MWH-2", "MWH-3"], n),
    "Quantity": rng.integers(50, 500, n),
})
print("shape:", df.shape)
print(df.head(), "\n")
print("columns:", list(df.columns))

ind = np.where(df.Quantity == max(df.Quantity))
print("heaviest quantity:", max(df.Quantity))
print("its location     :", df.Location[ind[0]].tolist())
print("its material     :", df.MaterialID[ind[0]].tolist())
```

OUTPUT

```
shape: (40, 3)
  MaterialID Location  Quantity
0       M102    MWH-3       126
1       M104    MWH-2        56
2       M102    MWH-1       168
3       M103    MWH-2       136
4       M102    MWH-3       343

columns: ['MaterialID', 'Location', 'Quantity']
heaviest quantity: 487
its location     : ['MWH-1']
its material     : ['M103']
```

🔵 8 · Selecting rows: iloc, loc and reset_index

Build a separate table for one warehouse:

```
ind = np.where(df.Location == 'MWH-2')
mydf = df.iloc[ind[0]]      # only the MWH-2 rows
```

Now there are two ways to pick rows, and the difference matters:

- iloc = by POSITION (1st row, 2nd row ...), always starts at 0 and counts the rows as they are now.
- loc = by LABEL — the number printed at the left of the row, i.e. the row's index label.

```
mydf.iloc[[0, 2]]     # the first and the third row of mydf
mydf.loc[[2, 8]]      # the rows whose LABEL is 2 and 8; error if these labels are not found
```

A student (Himanshu) spotted that line 106 of the class file should say loc, not iloc — exactly this difference. After filtering, the labels of mydf are the ORIGINAL row numbers of df (for example 3, 8, 11 ...), so mydf.iloc[[2, 8]] and mydf.loc[[2, 8]] are not the same thing.

The cure when you want labels and positions to agree:

```
mydf = df.iloc[ind[0]].reset_index(drop=True)
# labels now run 0, 1, 2 … so labels and positions match
```

drop=True throws away the old labels. Without it, the old index would be kept as an extra column.

▶ Try it — iloc vs loc after filtering

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(21)
n = 40
df = pd.DataFrame({
    "MaterialID": rng.choice(["M101", "M102", "M103", "M104"], n),
    "Location": rng.choice(["MWH-1", "MWH-2", "MWH-3"], n),
    "Quantity": rng.integers(50, 500, n),
})

ind = np.where(df.Location == "MWH-2")
mydf = df.iloc[ind[0]]
print("labels of mydf:", mydf.index.tolist()[:8], "...\n")
print("iloc[[0,2]] -> positions 0 and 2:\n", mydf.iloc[[0, 2]], "\n")
lab = mydf.index.tolist()[:2]
print("loc[%s] -> labels:\n" % lab, mydf.loc[lab], "\n")
try:
    mydf.loc[[1000]]
except KeyError:
    print("loc with a label that does not exist -> KeyError")

mydf = df.iloc[ind[0]].reset_index(drop=True)
print("after reset_index labels:", mydf.index.tolist()[:8], "...")
```

OUTPUT

```
labels of mydf: [1, 3, 5, 11, 12, 15, 16, 17] ...

iloc[[0,2]] -> positions 0 and 2:
   MaterialID Location  Quantity
```

```
1       M104    MWH-2        56
5       M103    MWH-2       464

loc[[1, 3]] -> labels:
   MaterialID Location  Quantity
1       M104    MWH-2        56
3       M103    MWH-2       136

loc with a label that does not exist -> KeyError
after reset_index labels: [0, 1, 2, 3, 4, 5, 6, 7] ...
```

🔵 9 · Removing and stacking rows

drop removes the rows with the given LABELS and returns a new DataFrame:

```
df2 = df.drop(ind[0])    # deletes the rows whose labels are in ind[0], saves the rest as df2
```

df itself is unchanged. The result df2 contains all rows EXCEPT MWH-2 — a quick way to say "everything except this location".

concat stacks tables on top of each other:

```
df1 = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='Pune')
df2 = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='Mumbai')
master_df = pd.concat([df1, df2]).reset_index(drop=True)
# puts all the rows of df2 under the rows of df1
# works when the number and names of columns match in both df1 and df2
```

Always follow concat with reset_index(drop=True), otherwise the two blocks each restart their labels at 0 and you get duplicate labels.

Merging by common columns (pd.merge) was the next topic — see the note for 15 and 16 October.

▶ Try it — drop and concat

```
import numpy as np
import pandas as pd

df = pd.DataFrame({"Location": ["A", "B", "A", "C", "B"], "Quantity": [10, 20, 30, 40, 50]})
ind = np.where(df.Location == "A")
print("drop the A rows:\n", df.drop(ind[0]), "\n")

d1 = pd.DataFrame({"Subject": ["Maths", "Stats"], "Grade": ["A", "B"]})
d2 = pd.DataFrame({"Subject": ["Python"], "Grade": ["A"]})
print("concat, labels not reset:\n", pd.concat([d1, d2]), "\n")
print("concat + reset_index(drop=True):\n", pd.concat([d1, d2]).reset_index(drop=True))
```

OUTPUT

```
drop the A rows:
   Location  Quantity
1        B        20
3        C        40
4        B        50

concat, labels not reset:
   Subject Grade
0   Maths     A
```

```
1   Stats     B
0  Python     A

concat + reset_index(drop=True):
   Subject Grade
0   Maths     A
1   Stats     B
2  Python     A
```

🟠 10 · Typical errors and fixes

- Forgetting the [0] in ind[0]. np.where returns a tuple; the positions are inside it.
- Writing a1 >= 4 & a1 <= 7 without brackets → error. Always (a1 >= 4) & (a1 <= 7).
- Using iloc with labels or loc with positions after filtering. Run reset_index(drop=True) after every filter you plan to index again.
- Reading a file that Python cannot find. Check os.getcwd() first.
- Using slicing end-points as inclusive. a1[2:5] stops at index 4.

🔵 11 · Practice questions

Use b = np.array([12, 5, 8, 20, 3, 15, 9, 7]).

Q1. What is b[2:6]? What is b[[0, 3, 7]]?

Q2. Write one line that gives the number of elements of b that are greater than 8.

Q3. Which indices have values that are less than 6 OR greater than 14?

Q4. What is the shape of b.reshape(-1, 2)?

Q5. A table t has 5 rows with labels 10, 11, 12, 13, 14. What does t.iloc[[1]] return and what does t.loc[[1]] return?

Q6. df has 200 rows. df.shape gives what? After t = df.iloc[np.where(df.Quantity > 400)[0]] and 14 rows qualify, what is t.shape if df has 6 columns?

Answers

A1. b[2:6] = [8, 20, 3, 15]; b[[0,3,7]] = [12, 20, 7].

A2. (b > 8).sum() → 4 (values 12, 20, 15, 9).

A3. np.where((b < 6) | (b > 14))[0] → indices 1 (5), 3 (20), 4 (3), 5 (15).

A4. (4, 2).

A5. iloc[[1]] returns the second row (label 11); loc[[1]] raises KeyError because label 1 does not exist.

A6. (200, 6); t.shape = (14, 6).

🟢 12 · One-page cheat sheet

```
np.array([..])                  make an array
a[i]   a[[i,j]]   a[i:j]        one, several, slice (stop excluded)
a.reshape(r, c)   reshape(-1, 1)  re-cut into rows and columns
np.where(cond)[0]               indices where the condition is True
(c1) & (c2)   (c1) | (c2)       AND, OR on arrays — brackets required
(cond).sum()                    how many True
```

```
a[cond]                          values where True
pd.read_excel(file, sheet_name=)  Excel sheet -> DataFrame
df.shape  head()  tail()  columns look at the table
df.iloc[[i,j]]   df.loc[[a,b]]    by position / by label
.reset_index(drop=True)          make labels 0,1,2…
df.drop(labels)                  remove rows
pd.concat([d1, d2])              stack tables
```

15–17 October — Combining, Cleaning & Reshaping Tables: merge, groupby, pivot

merge (joins) · Missing values · groupby · melt · pivot_table · crosstab · sort_values

These notes cover the second half of the pandas teaching by Sudeep Bhargava: how to join two tables, what to do when the join leaves holes, how to summarise by group, and how to turn a wide table into a tall one and back. The class used CDAC_DataBook.xlsx; the runnable cells here build tiny tables of the same shape so you can see every row and check the logic by hand.

How to read the grades: 🟢 start here · 🔵 core · 🟠 go deeper · 🔴 stretch

🟢 1 · Two ways to combine tables

- concat — STACK tables (rows under rows). Use it when the tables have the same columns, for example the Pune sheet and the Mumbai sheet of the same report. pd.concat([df1, df2]).reset_index(drop=True)
- merge — MATCH rows side by side using a common column (a key). Use it when the tables hold different facts about the same things, for example Salary in one table and Department in another, tied together by EmpID.

The faculty answer to the question "is this similar to SQL joins, sir?" was yes: pd.merge is a join. The words inner, outer, left and right mean exactly what they mean in SQL.

🔵 2 · pd.merge — choosing the key with on

Two small tables from the class data book. df1 is the Pune sheet, df2 the Mumbai sheet; each lists a Subject, its Grade and the trainer Name.

```
df1 = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='Pune')
df2 = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='Mumbai')
```

```
pd.merge(df1, df2, on='Subject')
pd.merge(df1, df2, on=['Subject', 'Grade'])
pd.merge(df1, df2, on=['Subject', 'Grade', 'Name'])
```

What each line asks:

- on='Subject' → keep subjects found in BOTH tables.
- on=['Subject','Grade'] → BOTH Subject AND Grade must match. A subject offered in both cities but with different grades disappears.
- on=['Subject','Grade','Name'] → Subject AND Grade AND Name must all match. If no row satisfies all three, you get an EMPTY DataFrame.
- If you leave on out, pandas uses ALL the column names the two tables have in common.

When the other columns have the same name in both tables (Grade and Name here), pandas keeps both and adds suffixes _x (left table) and _y (right table).

▶ Try it — merge with different keys

```
import pandas as pd

df1 = pd.DataFrame({"Subject": ["Maths", "Stats", "Python", "ML"],
                    "Grade":   ["A", "B", "A", "B"],
                    "Name":    ["Ravi", "Asha", "Kiran", "Meera"]})
df2 = pd.DataFrame({"Subject": ["Stats", "Python", "SQL", "Maths"],
                    "Grade":   ["B", "B", "A", "A"],
                    "Name":    ["Dev", "Neha", "Om", "Isha"]})

print("on Subject:\n", pd.merge(df1, df2, on="Subject"), "\n")
print("on Subject + Grade:\n", pd.merge(df1, df2, on=["Subject", "Grade"]), "\n")
print("on Subject + Grade + Name:\n", pd.merge(df1, df2, on=["Subject", "Grade", "Name"]))
```

OUTPUT

```
on Subject:
   Subject Grade_x Name_x Grade_y Name_y
0   Maths       A   Ravi       A   Isha
1   Stats       B   Asha       B    Dev
2  Python       A  Kiran       B   Neha

on Subject + Grade:
   Subject Grade Name_x Name_y
0   Maths     A   Ravi   Isha
1   Stats     B   Asha    Dev

on Subject + Grade + Name:
 Empty DataFrame
Columns: [Subject, Grade, Name]
Index: []
```

🔵 3 · The how option: inner, outer, left, right

```
pd.merge(df1, df2, on='Subject', how='inner')    # default
pd.merge(df1, df2, on='Subject', how='outer')
pd.merge(df1, df2, on='Subject', how='left')
pd.merge(df1, df2, on='Subject', how='right')
```

```
how       Keeps                                          Think of it as
inner     keys present in BOTH tables (the default)      AND
outer     keys present in EITHER table                   OR
left      every key of df1 (the table named first);      "df1 is the boss"
          df2-only keys do not appear
right     every key of df2 (the table named second);     "df2 is the boss"
          df1-only keys do not appear
```

Rows that exist on only one side get NaN (missing) in the other table's columns.

▶ Try it — inner, outer, left, right side by side

```
import pandas as pd

df1 = pd.DataFrame({"Subject": ["Maths", "Stats", "Python", "ML"],
                    "TrainerPune": ["Ravi", "Asha", "Kiran", "Meera"]})
df2 = pd.DataFrame({"Subject": ["Stats", "Python", "SQL", "Maths"],
```

```
                    "TrainerMumbai": ["Dev", "Neha", "Om", "Isha"]})

for how in ["inner", "outer", "left", "right"]:
    out = pd.merge(df1, df2, on="Subject", how=how)
    print("how = %s  (%d rows)" % (how, len(out)))
    print(out, "\n")
```

OUTPUT

```
how = inner  (3 rows)
  Subject TrainerPune TrainerMumbai
0   Maths        Ravi          Isha
1   Stats        Asha           Dev
2  Python       Kiran          Neha

how = outer  (5 rows)
  Subject TrainerPune TrainerMumbai
0      ML       Meera           NaN
1   Maths        Ravi          Isha
2  Python       Kiran          Neha
3     SQL         NaN            Om
4   Stats        Asha           Dev

how = left  (4 rows)
  Subject TrainerPune TrainerMumbai
0   Maths        Ravi          Isha
1   Stats        Asha           Dev
2  Python       Kiran          Neha
3      ML       Meera           NaN

how = right  (4 rows)
  Subject TrainerPune TrainerMumbai
0   Stats        Asha           Dev
1  Python       Kiran          Neha
2     SQL         NaN            Om
3   Maths        Ravi          Isha
```

🔵 4 · Merge on the shared columns, then handle the holes

On 16 October a Salary table and a Deptt table were merged. Both have Name and EmpID, so those two columns are the key automatically.

```
df1 = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='Salary')
df2 = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='Deptt')
pd.merge(df1, df2)                  # key = Name AND EmpID, how = inner
pd.merge(df1, df2, how='outer')     # key present in either table
```

With the outer merge:

- a person in df1 but not df2 → Deptt is NaN
- a person in df2 but not df1 → Salary is NaN

Find where Salary is missing, and split the table in two:

```
df3 = pd.merge(df1, df2, how='outer')
ind = np.where(df3.Salary.isnull())      # indices of rows with a missing Salary
df3B = df3.iloc[ind[0]]                  # all rows with a missing Salary
df3A = df3.drop(ind[0])                  # rows with NO missing Salary
```

🔵 5 · Filling missing values with fillna

Four fillers were demonstrated:

```
df3.Salary = df3.Salary.fillna(df3.Salary.median())   # median of the column
df3.Salary = df3.Salary.fillna(60)                    # a fixed number
df3.Deptt = df3.Deptt.fillna(df3.Deptt.mode()[0])     # the most frequent label (mode)
df3.Deptt = df3.Deptt.fillna('Overheads')             # a fixed label
```

How to choose:

- Numeric column → median (or mean). The median is preferred because it is NOT pulled by outliers; the mean is.
- Text/category column → mode (most common label), or an agreed label such as 'Overheads' that explains why the value is missing.
- Fixed number such as 60 → only when the business has a rule for it.
- mode() returns a Series (there can be ties), so you take the first with [0].

▶ Try it — outer merge, find the holes, fill them

```
import numpy as np
import pandas as pd

sal = pd.DataFrame({"Name": ["Amit", "Beena", "Chetan", "Divya"],
                    "EmpID": [101, 102, 103, 104],
                    "Salary": [50, 70, 60, 80]})
dep = pd.DataFrame({"Name": ["Amit", "Beena", "Eshan", "Farah"],
                    "EmpID": [101, 102, 105, 106],
                    "Deptt": ["IT", "HR", "IT", "IT"]})

print("inner:\n", pd.merge(sal, dep), "\n")
df3 = pd.merge(sal, dep, how="outer")
print("outer:\n", df3, "\n")

ind = np.where(df3.Salary.isnull())
print("rows with missing Salary:", ind[0])
df3B = df3.iloc[ind[0]]
df3A = df3.drop(ind[0])
print("df3A (complete):", len(df3A), "rows;  df3B (missing):", len(df3B), "rows\n")

df3["Salary"] = df3.Salary.fillna(df3.Salary.median())
df3["Deptt"] = df3.Deptt.fillna(df3.Deptt.mode()[0])
print("after filling with median and mode:\n", df3)
```

OUTPUT

```
inner:
     Name  EmpID  Salary Deptt
0   Amit    101      50    IT
1  Beena    102      70    HR

outer:
      Name  EmpID  Salary Deptt
0    Amit    101    50.0    IT
1   Beena    102    70.0    HR
2  Chetan    103    60.0   NaN
3   Divya    104    80.0   NaN
4   Eshan    105     NaN    IT
5   Farah    106     NaN    IT
```

```
rows with missing Salary: [4 5]
df3A (complete): 4 rows;  df3B (missing): 2 rows

after filling with median and mode:
      Name  EmpID  Salary Deptt
0    Amit    101    50.0    IT
1   Beena    102    70.0    HR
2  Chetan    103    60.0    IT
3   Divya    104    80.0    IT
4   Eshan    105    65.0    IT
5   Farah    106    65.0    IT
```

🔵 6 · groupby — one answer per group

Question: what is the total Quantity for each MaterialID? The recipe is SPLIT the table into groups, APPLY a calculation, COMBINE the answers.

```
grp1 = df.groupby('MaterialID')     # creates groups based on MaterialID
grp1.groups                         # a dictionary: key = a MaterialID, value = list of row
indices
```

Then ask for a summary of one column:

```
grp1.Quantity.sum().reset_index()      # total Quantity of every MaterialID
grp1.Quantity.count().reset_index()    # number of rows (consignments) per MaterialID
grp1.Quantity.mean().reset_index()     # average Quantity per MaterialID
```

reset_index() turns the group labels from the index into an ordinary column. A student asked "what if I don't write reset_index()?" Nothing breaks: you still get the answer, but the MaterialID is the row label instead of a column; add reset_index() when you want a normal table to merge, plot or save.

Several summaries at once with agg:

```
df1 = grp1.Quantity.agg(['size', 'sum', 'mean']).reset_index()
# 4 columns: MaterialID, size (count), sum, mean
```

▶ Try it — groupby on one key

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(21)
n = 40
df = pd.DataFrame({
    "MaterialID": rng.choice(["M101", "M102", "M103", "M104"], n),
    "Location": rng.choice(["MWH-1", "MWH-2", "MWH-3"], n),
    "Quantity": rng.integers(50, 500, n),
})

grp1 = df.groupby("MaterialID")
print("groups -> first 3 indices of each:")
for k, v in grp1.groups.items():
    print("  ", k, list(v)[:3])
print()
```

```
print(grp1.Quantity.sum().reset_index(), "\n")
print(grp1.Quantity.agg(["size", "sum", "mean"]).reset_index())
```

OUTPUT

```
groups -> first 3 indices of each:
   M101 [7, 12, 15]
   M102 [0, 2, 4]
   M103 [3, 5, 9]
   M104 [1, 11, 14]

  MaterialID  Quantity
0       M101      1586
1       M102      2532
2       M103      2758
3       M104      2872

  MaterialID  size   sum        mean
0       M101     7  1586  226.571429
1       M102    11  2532  230.181818
2       M103     9  2758  306.444444
3       M104    13  2872  220.923077
```

🔵 7 · groupby with two keys

```
grp2 = df.groupby(['MaterialID', 'Location'])
grp2.groups
grp2.Quantity.count().reset_index()
df2 = grp2.Quantity.agg(['size', 'sum', 'mean']).reset_index()
```

```
grp3 = df.groupby('Location')
grp4 = df.groupby(['Location', 'MaterialID'])
```

The order of the keys matters for how the result is arranged. The first key is the PRIMARY key; it changes only after every possible value of the secondary key has been used. Result of groupby(['MaterialID','Location']): all locations for M101, then all locations for M102, and so on. Swap the keys and you get all materials of MWH-1, then all of MWH-2.

Himanshu's question: "Can we use np.where with groupby, like GROUP BY … HAVING in SQL?" The idea is exactly right. First summarise, then filter the SUMMARY table:

```
summary = grp2.Quantity.agg(['size', 'sum']).reset_index()
summary[summary['sum'] > 1000]         # HAVING sum > 1000
# or  summary.iloc[np.where(summary['sum'] > 1000)[0]]
```

▶ Try it — two-key groupby and a HAVING-style filter

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(21)
n = 40
df = pd.DataFrame({
    "MaterialID": rng.choice(["M101", "M102", "M103", "M104"], n),
    "Location": rng.choice(["MWH-1", "MWH-2", "MWH-3"], n),
    "Quantity": rng.integers(50, 500, n),
```

```
})

grp2 = df.groupby(["MaterialID", "Location"])
summary = grp2.Quantity.agg(["size", "sum", "mean"]).reset_index()
print(summary.head(8), "\n")
print("HAVING sum > 1000:")
print(summary[summary["sum"] > 1000])
```

OUTPUT

```
  MaterialID Location  size   sum        mean
0       M101    MWH-1     3   891  297.000000
1       M101    MWH-2     3   355  118.333333
2       M101    MWH-3     1   340  340.000000
3       M102    MWH-1     3   606  202.000000
4       M102    MWH-2     2   594  297.000000
5       M102    MWH-3     6  1332  222.000000
6       M103    MWH-1     4  1172  293.000000
7       M103    MWH-2     4  1104  276.000000

HAVING sum > 1000:
   MaterialID Location  size   sum   mean
5        M102    MWH-3     6  1332  222.0
6        M103    MWH-1     4  1172  293.0
7        M103    MWH-2     4  1104  276.0
11       M104    MWH-3     4  1126  281.5
```

🔵 8 · Wide table → tall table: melt

A WIDE table has one column per year (or month, or product). A TALL table has one column that says which year and one that holds the value. Most analysis and plotting wants the tall form.

The Health sheet is wide: Country, Disease, then a column for each year.

```
df = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='Health')
df1 = pd.melt(df, id_vars=['Country', 'Disease'], var_name='Year', value_name='Cases')
```

- id_vars: the columns that stay as they are (the identifiers)
- var_name: the NEW column that will hold the old column names (the years)
- value_name: the NEW column that will hold the numbers that were under those columns

▶ Try it — melt

```
import pandas as pd

df = pd.DataFrame({"Country": ["India", "India", "Kenya", "Kenya"],
                   "Disease": ["Malaria", "Dengue", "Malaria", "Dengue"],
                   "2019": [100, 40, 300, 10],
                   "2020": [90, 55, 280, 12],
                   "2021": [70, 60, 250, 15]})
print("WIDE:\n", df, "\n")
df1 = pd.melt(df, id_vars=["Country", "Disease"], var_name="Year", value_name="Cases")
print("TALL:", df1.shape, "\n")
print(df1)
```

OUTPUT

```
WIDE:
   Country  Disease  2019  2020  2021
0   India  Malaria   100    90    70
1   India   Dengue    40    55    60
2   Kenya  Malaria   300   280   250
3   Kenya   Dengue    10    12    15

TALL: (12, 4)

   Country  Disease  Year  Cases
0    India  Malaria  2019    100
1    India   Dengue  2019     40
2    Kenya  Malaria  2019    300
3    Kenya   Dengue  2019     10
4    India  Malaria  2020     90
5    India   Dengue  2020     55
6    Kenya  Malaria  2020    280
7    Kenya   Dengue  2020     12
8    India  Malaria  2021     70
9    India   Dengue  2021     60
10   Kenya  Malaria  2021    250
11   Kenya   Dengue  2021     15
```

🔵 9 · Tall table → wide table: pivot_table

The reverse journey. A student asked "so converting tall to broad is pivot_table, right?" Yes.

```
df2 = df1.pivot_table(index=['Country', 'Year'], columns='Disease', values='Cases')
df2 = df2.reset_index().rename_axis(columns=None)
```

- index: what stays down the left side
- columns: the column whose DIFFERENT VALUES become NEW COLUMNS (one per unique value)
- values: the numbers that fill the cells
- reset_index() and rename_axis(columns=None) clean up the labels so you get a plain table back

On the board: df → df1 converted a broad DataFrame to a tall one; df1 → df2 converted a tall DataFrame to a broad one, with one column for each unique Disease.

▶ Try it — melt then pivot_table (round trip)

```
import pandas as pd

df = pd.DataFrame({"Country": ["India", "India", "Kenya", "Kenya"],
                   "Disease": ["Malaria", "Dengue", "Malaria", "Dengue"],
                   "2019": [100, 40, 300, 10],
                   "2020": [90, 55, 280, 12],
                   "2021": [70, 60, 250, 15]})
df1 = pd.melt(df, id_vars=["Country", "Disease"], var_name="Year", value_name="Cases")
df2 = (df1.pivot_table(index=["Country", "Year"], columns="Disease", values="Cases")
          .reset_index().rename_axis(columns=None))
print(df2)
```

OUTPUT

```
  Country  Year  Dengue  Malaria
0   India  2019    40.0    100.0
```

```
1   India  2020    55.0     90.0
2   India  2021    60.0     70.0
3   Kenya  2019    10.0    300.0
4   Kenya  2020    12.0    280.0
5   Kenya  2021    15.0    250.0
```

🔵 10 · pivot_table as a summary tool, and crosstab

pivot_table also summarises. Total quantity for every Location × MaterialID:

```
df1 = df.pivot_table(index='Location', columns='MaterialID', values='Quantity').reset_index()
# default aggfunc is the MEAN (not the sum!)
```

```
df1 = df.pivot_table(index='Location', columns='MaterialID', values='Quantity',
                     aggfunc='sum').reset_index().rename_axis(columns=None)
```

Options of aggfunc:

- sum → total
- count → frequency
- mean → average
- std → standard deviation
- var → variance

Cells with no matching rows are NaN. Aniket asked whether NaN can be replaced with 0 — yes, fill_value=0. margins=True adds the totals row and column (labelled All):

```
df1 = df.pivot_table(index='Location', columns='MaterialID', values='Quantity',
                     aggfunc='sum', fill_value=0, margins=True)
```

crosstab is the shortcut when you only want counts (or one sum) of two categorical columns:

```
pd.crosstab(df.MaterialID, df.Location, values=df.Quantity, aggfunc='sum')
pd.crosstab(df.MaterialID, df.Location)          # with no values: number of rows
```

▶ Try it — pivot_table options and crosstab

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(21)
n = 40
df = pd.DataFrame({
    "MaterialID": rng.choice(["M101", "M102", "M103", "M104"], n),
    "Location": rng.choice(["MWH-1", "MWH-2", "MWH-3"], n),
    "Quantity": rng.integers(50, 500, n),
})

print("default aggfunc (mean):\n", df.pivot_table(index="Location", columns="MaterialID",
values="Quantity").round(0), "\n")
p = df.pivot_table(index="Location", columns="MaterialID", values="Quantity",
                   aggfunc="sum", fill_value=0, margins=True)
print("sum with totals:\n", p, "\n")
print("crosstab counts:\n", pd.crosstab(df.MaterialID, df.Location))
```

OUTPUT

```
default aggfunc (mean):
 MaterialID   M101   M102   M103   M104
Location
MWH-1       297.0  202.0  293.0  189.0
MWH-2       118.0  297.0  276.0  198.0
MWH-3       340.0  222.0  482.0  282.0

sum with totals:
 MaterialID  M101  M102  M103  M104   All
Location
MWH-1        891   606  1172   755  3424
MWH-2        355   594  1104   991  3044
MWH-3        340  1332   482  1126  3280
All         1586  2532  2758  2872  9748

crosstab counts:
 Location    MWH-1  MWH-2  MWH-3
MaterialID
M101            3      3      1
M102            3      2      6
M103            4      4      1
M104            4      5      4
```

🔵 11 · Sorting with sort_values

```
df_sorted = df.sort_values('Quantity').reset_index(drop=True)                    # increasing
df_sorted = df.sort_values('Quantity', ascending=False).reset_index(drop=True)   # decreasing
```

Sorting on several columns, each with its own direction:

```
df_sorted = df.sort_values(['MaterialID', 'Location', 'Quantity'],
                           ascending=[True, False, False])
# first level : MaterialID ascending
# second level: Location descending
# third level : Quantity descending
```

Sagar asked "can we sort count-wise?" Yes: sort the counts table, not the labels: counts.sort_values('Counts', ascending=False). This is used for bar charts in the visualisation note.

▶ Try it — multi-level sort

```
import pandas as pd
df = pd.DataFrame({"MaterialID": ["M2", "M1", "M2", "M1", "M2"],
                   "Location": ["A", "B", "B", "A", "B"],
                   "Quantity": [30, 10, 50, 20, 40]})
print(df.sort_values(["MaterialID", "Location", "Quantity"], ascending=[True, False, False]))
```

OUTPUT

```
  MaterialID Location  Quantity
1         M1        B        10
3         M1        A        20
2         M2        B        50
```

```
4         M2        B         40
0         M2        A         30
```

🟠 12 · Common mistakes

- Using concat when you meant merge (or the other way round). concat stacks rows, merge matches rows.
- Merging on a key with duplicate values on both sides: every pair is matched, so the result can have MORE rows than either table.
- Expecting inner to keep unmatched rows. Only outer/left/right keep them.
- Forgetting that pivot_table's default is the MEAN. Always state aggfunc.
- Filling every missing number with the mean when there are outliers. The median is safer.
- Using df.drop(ind[0]) after the labels have been shuffled by a sort — the numbers in ind refer to LABELS, so reset_index(drop=True) first.

🔵 13 · Practice questions

Table P (Pune): Subject = Maths, Stats, Python; Table M (Mumbai): Subject = Stats, SQL, Python.

Q1. How many rows has the merge on Subject with how='inner'? With 'outer'? With 'left' (P first)? With 'right'?

Q2. In an outer merge, which columns hold NaN for SQL?

Q3. Salary column [50, 70, NaN, 80, NaN]. What does fillna(median) put in the blanks? What would the mean have put?

Q4. Which function turns a table with columns 2019, 2020, 2021 into one Year column and one Cases column? Which one reverses it?

Q5. Write the call for: total Quantity for each Location and MaterialID, as an ordinary table.

Q6. You run df.pivot_table(index='Location', columns='MaterialID', values='Quantity') and get averages. How do you get totals, with 0 instead of NaN?

Answers

A1. inner 2 (Stats, Python); outer 4 (Maths, Stats, Python, SQL); left 3 (Maths, Stats, Python); right 3 (Stats, SQL, Python).

A2. The columns that come from table P (they are missing for SQL).

A3. Median of 50, 70, 80 = 70. Mean = 66.7, shown as 66.67.

A4. melt; pivot_table.

A5. df.groupby(['Location','MaterialID']).Quantity.sum().reset_index()

A6. add aggfunc='sum', fill_value=0.

🟢 14 · One-page cheat sheet

```
pd.concat([d1, d2]).reset_index(drop=True)      stack rows
pd.merge(d1, d2, on=[keys], how='inner')        join (inner/outer/left/right)
d.isnull()   np.where(d.col.isnull())           find missing values
d.col.fillna(value)                             fill (median / mode / fixed)
```

```
d.groupby(keys).col.agg(['size','sum','mean'])  one answer per group
.reset_index()                                  group labels back to a column
pd.melt(d, id_vars, var_name, value_name)       wide -> tall
d.pivot_table(index, columns, values, aggfunc)  tall -> wide / summary table
pd.crosstab(a, b)                               counts of two categories
d.sort_values([cols], ascending=[..])           sort
```

17–21 October — Data Visualisation: Which Chart for Which Data

Bar · Pie · Histogram · Box plot · Grouped and stacked bars · Scatter · Multi-variable plots

Sudeep Bhargava taught charts by one organising idea: look at how many variables you are plotting (one, two or three) and what TYPE each is (categorical or continuous). That tells you the chart. These notes follow that order. The ERPData sheet (MaterialID, Location, Quantity) supplies the categorical examples; the faithful sheet (eruptions, waiting) supplies the continuous ones; Marks.xlsx was used for grouped box plots.

Important: the plotting libraries (matplotlib, seaborn) do not run inside the ASCEND browser, so the plotting code below is shown as code for your notebook (VS Code or Colab). The runnable cells do all the calculations behind the charts and draw simple text versions, so you can check what the chart should look like.

How to read the grades: 🟢 start here · 🔵 core · 🟠 go deeper · 🔴 stretch

Libraries used by every cell in your notebook:

```
import numpy as np
import pandas as pd
import matplotlib
from matplotlib import pyplot as plt
import seaborn as sns
```

🟢 1 · The chart-choice map (the 21 October summary)

```
Variables      Data types                        Chart
UNIVARIATE (one variable)
               categorical                       bar chart (frequency or sum) or pie chart
(proportions)
               continuous                        histogram (counts inside intervals) or box plot
(spread + outliers)
BIVARIATE (two variables)
               2 categorical                     grouped or stacked bar chart
               1 categorical + 1 continuous      multiple box plots (one per category)
               2 continuous                      scatter plot
MULTIVARIATE (three or more)
               2 continuous + 1 categorical      grouped scatter plot (colour by category)
               1 continuous + 2 categorical      grouped box plot
```

Keep this table open while you work. Pick the row first, the code second.

🟢 2 · Univariate, categorical: bar chart and pie chart

Step 1 — turn the column into a table of counts:

```
np.unique(df.MaterialID)                        # the distinct values, sorted
df_counts = df.MaterialID.value_counts().reset_index()
df_counts.columns = ['MaterialID', 'Counts']     # give the two columns clear names
```

Step 2 — order the bars. Two sensible orders:

```
df_sorted = df_counts.sort_values('MaterialID')                          # A to Z on the x axis
df_counts = df_counts.sort_values('Counts', ascending=False)           # tallest bar first
```

The second order answers Sagar's question "can we sort count-wise?" — yes, sort the counts table by Counts.

Step 3 — draw:

```
plt.bar(df_sorted.MaterialID, df_sorted.Counts)                 # MaterialIDs in ascending order
on x
plt.bar(df_counts.MaterialID, df_counts.Counts)                 # tallest bar first
sns.barplot(x=df_sorted.MaterialID, y=df_sorted.Counts)         # seaborn version
plt.pie(labels=df_counts.MaterialID, x=df_counts.Counts)        # pie chart
```

Bar chart → compare AMOUNTS across categories. Pie chart → show each category's SHARE of a whole; keep it to a few slices.

▶ Try it — The numbers behind a bar chart (value counts and a text bar chart)

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(21)
df = pd.DataFrame({"MaterialID": rng.choice(["M101", "M102", "M103", "M104"], 60, p=[.15,
.30, .25, .30]),
                   "Location": rng.choice(["MWH-1", "MWH-2", "MWH-3"], 60),
                   "Quantity": rng.integers(50, 500, 60)})

print("unique values:", np.unique(df.MaterialID))
counts = df.MaterialID.value_counts().reset_index()
counts.columns = ["MaterialID", "Counts"]

print("\nalphabetical order:")
for _, r in counts.sort_values("MaterialID").iterrows():
    print("  %-5s %s %d" % (r.MaterialID, "#" * r.Counts, r.Counts))
print("\ntallest first:")
for _, r in counts.sort_values("Counts", ascending=False).iterrows():
    print("  %-5s %s %d" % (r.MaterialID, "#" * r.Counts, r.Counts))
print("\nshare of each (what the pie chart shows):")
print((counts.set_index("MaterialID").Counts / counts.Counts.sum() * 100).round(1))
```

OUTPUT

```
unique values: ['M101' 'M102' 'M103' 'M104']

alphabetical order:
  M101  ######### 9
  M102  #################### 20
  M103  ############### 15
  M104  ################ 16

tallest first:
  M102  #################### 20
  M104  ################ 16
  M103  ############### 15
  M101  ######### 9

share of each (what the pie chart shows):
```

```
MaterialID
M102    33.3
M104    26.7
M103    25.0
M101    15.0
Name: Counts, dtype: float64
```

🔵 3 · Univariate, continuous: the histogram — and how it differs from a bar chart

A histogram counts how many values fall inside each INTERVAL (bin) of a continuous variable.

```
df = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='faithful')
erupt = df.eruptions
wait = df.waiting
plt.hist(erupt)
```

The differences written on the board (six statements, three pairs):

```
Bar chart                                   Histogram
x-axis variable is categorical              x-axis variable is continuous
bars do not touch each other                bars touch each other
bars have equal thickness                   bars may have unequal thickness (bins of different
widths)
```

Choosing the bins yourself:

```
my_cats = [1.5, 2.4, 3.6, 4.2, 5.5]          # the bin edges: 1.5–2.4, 2.4–3.6, 3.6–4.2, 4.2–5.5
plt.hist(erupt, bins=my_cats)
plt.hist(erupt, bins=my_cats, color='black')
sns.histplot(erupt, color='black', bins=my_cats)       # seaborn, univariate
```

🟠 With unequal bins, taller does not always mean "more values per unit": a wide bin collects more values simply because it is wide. If you want the AREA of a bar to represent frequency, use density=True (a stretch topic).

▶ Try it — Histogram counts inside bins (what plt.hist computes)

```
import numpy as np

rng = np.random.default_rng(8)
# a two-humped shape like Old Faithful: short and long eruptions
erupt = np.concatenate([rng.normal(2.0, 0.30, 100), rng.normal(4.3, 0.40, 172)])

def show(edges):
    counts, edges = np.histogram(erupt, bins=edges)
    for i in range(len(counts)):
        print("  %4.1f - %4.1f | %-45s %d" % (edges[i], edges[i+1], "#" * (counts[i] // 2),
counts[i]))

print("default 10 equal bins:")
show(10)
print("\nbins chosen by us (my_cats), unequal widths:")
show([1.5, 2.4, 3.6, 4.2, 5.5])
```

OUTPUT

```
default 10 equal bins:
   1.3 -  1.7 | ########                                  16
   1.7 -  2.1 | ####################                      45
   2.1 -  2.5 | ##############                            29
   2.5 -  2.9 | #####                                     10
   2.9 -  3.3 | #                                         2
   3.3 -  3.7 | ###                                       7
   3.7 -  4.1 | ##################                        39
   4.1 -  4.5 | ##############################            66
   4.5 -  4.9 | ###################                       40
   4.9 -  5.3 | #########                                 18

bins chosen by us (my_cats), unequal widths:
   1.5 -  2.4 | ######################################    79
   2.4 -  3.6 | ###########                               22
   3.6 -  4.2 | ##############################            63
   4.2 -  5.5 | ################################################## 102
```

🔵 4 · Comparing groups with histograms: split, then overlay

Question: do eruptions that follow a long wait look different from those that follow a short wait? Split the data at the median of the waiting time:

```
grpA = erupt[np.where(wait >= np.median(wait))[0]]
grpB = erupt[np.where(wait <  np.median(wait))[0]]
plt.hist(grpA, bins=my_cats, color='blue',  label='Above 50% waiting time')
plt.hist(grpB, bins=my_cats, color='black', label='Below 50% waiting time')
```

Seaborn can do the same with a category column. Build the column Category with A / B:

```
code = []
for ctr in erupt:
    if ctr >= np.median(erupt):
        code.append('A')
    else:
        code.append('B')
df['Category'] = code
sns.histplot(x='eruptions', hue='Category', data=df, bins=my_cats)
```

The loop works, but there is a shorter way (shown in the cell below): np.where(erupt >= np.median(erupt), 'A', 'B').

▶ Try it — Build a Category column two ways

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(8)
erupt = pd.Series(np.concatenate([rng.normal(2.0, 0.30, 100), rng.normal(4.3, 0.40, 172)]))

code = []
for ctr in erupt:                       # the loop from class
    if ctr >= np.median(erupt):
        code.append("A")
    else:
        code.append("B")
```

```
short = np.where(erupt >= np.median(erupt), "A", "B")     # one line, same result
print("same answer?", list(short) == code)
df = pd.DataFrame({"eruptions": erupt, "Category": code})
print(df.Category.value_counts())
```

OUTPUT

```
same answer? True
Category
B    136
A    136
Name: count, dtype: int64
```

🔵 5 · Univariate, continuous: the box plot and its outlier fences

A box plot packs five numbers into one picture: minimum, Q1 (25th percentile), median, Q3 (75th percentile), maximum. The box runs Q1 to Q3; the line inside is the median; whiskers extend towards the extremes; dots beyond the whiskers are outliers.

```
plt.boxplot(erupt)        # matplotlib
sns.boxplot(erupt)        # seaborn (with newer seaborn write sns.boxplot(x=erupt))
```

The main purpose is to find outliers. The fences used by the plot, and the indices of the points beyond them:

```
# IQR = Q3 - Q1
# UF  = Q3 + 1.5 * IQR   (upper fence)
# LF  = Q1 - 1.5 * IQR   (lower fence)
UF = np.percentile(erupt, 75) + 1.5*(np.percentile(erupt, 75) - np.percentile(erupt, 25))
LF = np.percentile(erupt, 25) - 1.5*(np.percentile(erupt, 75) - np.percentile(erupt, 25))
ind_upper = np.where(erupt > UF)[0]
ind_lower = np.where(erupt < LF)[0]
```

To demonstrate, two artificial outliers were added to the data:

```
mydata = pd.DataFrame([[9, 120, 'A'], [15, 140, 'B']], columns=df.columns)
df = pd.concat([df, mydata]).reset_index(drop=True)
erupt = df.eruptions          # <- re-run this line!
plt.boxplot(erupt)
```

A student's error that cost the class ten minutes: after the concat, the old variable erupt still pointed at the OLD column. Whenever df changes, re-create erupt (and wait, and Category) from the new df.

Percentiles: np.percentile(erupt, 75) is Q3 — the value below which 75% of the data lie — and np.percentile(erupt, 25) is Q1. Their difference is the IQR, not either one on its own (Debmalya's question). The full story of the 1.5 multiplier is in the 5 September note on the IQR method.

▶ Try it — Outlier fences after adding two extreme rows

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(8)
erupt = np.concatenate([rng.normal(2.0, 0.30, 100), rng.normal(4.3, 0.40, 172)])
waiting = rng.normal(70, 13, 272).round()
df = pd.DataFrame({"eruptions": erupt, "waiting": waiting, "Category": "A"})
```

```
mydata = pd.DataFrame([[9, 120, "A"], [15, 140, "B"]], columns=df.columns)
df = pd.concat([df, mydata]).reset_index(drop=True)
erupt = df.eruptions                        # re-create after the concat!

q1, q3 = np.percentile(erupt, 25), np.percentile(erupt, 75)
iqr = q3 - q1
UF, LF = q3 + 1.5*iqr, q1 - 1.5*iqr
print("Q1 = %.2f  Q3 = %.2f  IQR = %.2f" % (q1, q3, iqr))
print("lower fence = %.2f   upper fence = %.2f" % (LF, UF))
ind_upper = np.where(erupt > UF)[0]
ind_lower = np.where(erupt < LF)[0]
print("outliers above:", ind_upper, "values", erupt[ind_upper].tolist())
print("outliers below:", ind_lower)
```

OUTPUT

```
Q1 = 2.16  Q3 = 4.41  IQR = 2.26
lower fence = -1.23   upper fence = 7.80
outliers above: [272 273] values [9.0, 15.0]
outliers below: []
```

🔵 6 · Bivariate: two categorical variables → grouped or stacked bars

Question: how many consignments of each MaterialID went to each Location? First build the table of counts, then plot it:

```
dfnew = df.pivot_table(index='MaterialID', columns='Location', values='Quantity',
                       aggfunc='count').reset_index().rename_axis(columns=None)
dfnew.plot(kind='bar')                                  # grouped bars: one bar per Location
inside each MaterialID
```

Without reset_index(), the labels stay as the index and the x axis shows the MaterialIDs correctly:

```
dfnew = df.pivot_table(index='MaterialID', columns='Location', values='Quantity',
                       aggfunc='count').rename_axis(columns=None)
dfnew.plot(kind='bar')
```

The stacked version puts the locations on top of each other, so the total height shows the total count of the material:

```
dfnew = df.pivot_table(index='MaterialID', columns='Location', values='Quantity',
                       aggfunc='count').reset_index().rename_axis(columns=None)
dfnew.plot.bar(x='MaterialID', y=dfnew.columns[1:6], stacked=True)
```

The same pictures with seaborn:

```
sns.barplot(x='MaterialID', y='Quantity', hue='Location', data=df, ci=None, estimator=np.size)
sns.histplot(data=df, x='MaterialID', hue='Location', multiple='stack')
```

In the barplot, estimator=np.size means "the height of each bar = the number of rows" instead of the default mean of Quantity. hue='Location' makes one bar per location.

🟠 Newer seaborn versions replace ci=None with errorbar=None.

Choosing grouped vs stacked: grouped bars are best to COMPARE the parts across categories; stacked bars are best to show the TOTAL per category and how it is made up.

▶ Try it — The table behind a grouped / stacked bar chart

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(21)
df = pd.DataFrame({"MaterialID": rng.choice(["M101", "M102", "M103", "M104"], 60),
                   "Location": rng.choice(["MWH-1", "MWH-2", "MWH-3"], 60),
                   "Quantity": rng.integers(50, 500, 60)})

dfnew = (df.pivot_table(index="MaterialID", columns="Location", values="Quantity",
aggfunc="count")
           .rename_axis(columns=None))
print(dfnew, "\n")
print("stacked height (= row total) for each MaterialID:")
for m, tot in dfnew.sum(axis=1).items():
    print("  %-5s %s %d" % (m, "#" * int(tot), tot))
```

OUTPUT

```
            MWH-1  MWH-2  MWH-3
MaterialID
M101            5      3      3
M102            9      3      4
M103            5      6      2
M104            7      8      5

stacked height (= row total) for each MaterialID:
  M101  ########### 11
  M102  ################ 16
  M103  ############# 13
  M104  ################### 20
```

🔵 7 · Bivariate: one categorical + one continuous → box plot per category

Compare the distribution of a number across groups. Using the Category column built earlier:

```
dfA = df.iloc[np.where(df.Category == 'A')[0]]
dfB = df.iloc[np.where(df.Category == 'B')[0]]
plt.boxplot([dfA.eruptions, dfB.eruptions])          # matplotlib: pass a list of the groups
sns.boxplot(x='Category', y='eruptions', data=df)    # seaborn: names the columns
```

The plot shows at a glance whether the groups have different medians, different spreads or different outliers. This is the picture that comes BEFORE a 2-sample test (see the note of 6–8 October).

One continuous + two categorical (multivariate) — a grouped box plot with hue:

```
df = pd.read_excel('Marks.xlsx')
sns.boxplot(x='Subject', y='Marks', hue='Trainer', data=df)
sns.boxplot(x='Trainer', y='Marks', hue='Subject', data=df)
```

The two lines contain the same variables but tell different stories: the first asks "how do the trainers compare WITHIN each subject?", the second asks "how do the subjects compare within each trainer?". The variable on the x axis is the one you want to compare along; the hue variable is the context.

▶ Try it — Box-plot numbers for two groups

```
import numpy as np

rng = np.random.default_rng(8)
groupA = rng.normal(4.2, 0.6, 130)
groupB = rng.normal(2.4, 0.7, 142)
for name, g in [("A", groupA), ("B", groupB)]:
    q1, med, q3 = np.percentile(g, [25, 50, 75])
    print("Category %s: n=%d  min=%.2f  Q1=%.2f  median=%.2f  Q3=%.2f  max=%.2f" %
          (name, len(g), g.min(), q1, med, q3, g.max()))
```

OUTPUT

```
Category A: n=130  min=2.72  Q1=3.73  median=4.16  Q3=4.59  max=5.72
Category B: n=142  min=0.19  Q1=1.91  median=2.40  Q3=2.90  max=4.07
```

🔵 8 · Bivariate and multivariate with scatter plots

Two continuous variables: scatter plot — one point per row.

```
plt.scatter(wait, erupt)               # matplotlib: first argument is x
sns.scatterplot(y=wait, x=erupt)       # seaborn as typed in class: note y= and x= are named
```

Careful with the axes. In the first line waiting is on the x axis; in the second line, as written, erupt is on x and wait on y — the picture is flipped. Decide which variable is the explanatory one (x) and which is the response (y) before you type.

A third variable (categorical) gets a colour — this is the multivariate scatter plot:

```
sns.relplot(x='waiting', y='eruptions', hue='Category', data=df)
```

How to read a scatter plot: direction (up = positive relationship, down = negative), tightness (close to a line = strong correlation), clusters (separate groups), and isolated points (possible outliers). The number behind the picture is the Pearson correlation of the 6–8 October note.

▶ Try it — The number behind a scatter plot

```
import numpy as np
from scipy.stats import pearsonr

rng = np.random.default_rng(42)
waiting = rng.normal(70, 13, 272).round()
eruptions = -1.87 + 0.0756 * waiting + rng.normal(0, 0.4, 272)
r, p = pearsonr(waiting, eruptions)
print("correlation r = %.2f" % r)

cat = np.where(eruptions >= np.median(eruptions), "A", "B")
for c in ["A", "B"]:
    print("Category %s: mean waiting = %.1f, mean eruptions = %.2f" %
          (c, waiting[cat == c].mean(), eruptions[cat == c].mean()))
```

OUTPUT

```
correlation r = 0.91
Category A: mean waiting = 77.9, mean eruptions = 4.15
Category B: mean waiting = 61.0, mean eruptions = 2.61
```

🟠 9 · Mistakes and fixes seen in class

- NameError or a wrong plot after changing df. Re-create erupt, wait and Category from the new df.
- Using a bar chart for a continuous variable (or a histogram for a category). Check the type first.
- Pie charts with many slices — unreadable. Use a sorted bar chart.
- Forgetting to name the columns after value_counts().reset_index() (the names differ between pandas versions). Set them yourself: df_counts.columns = ['MaterialID', 'Counts'].
- Swapped x and y in scatter plots.
- Reading unequal-width histogram bars as if they all covered equal intervals.

🔵 10 · Practice questions

Which chart would you use for each of these?

Q1. Share of tickets by priority (P1, P2, P3).

Q2. Distribution of delivery times in minutes.

Q3. Number of consignments per location, split by material.

Q4. Salary of staff in each department, to see spread and outliers.

Q5. Height vs weight of 100 people, coloured by gender.

Q6. Marks of students by subject, split by trainer.

Q7. Quartiles are Q1 = 20, Q3 = 50. What are the fences? A value of 100 — is it an outlier?

Answers

A1. Pie chart (shares of a whole); a sorted bar chart also works.

A2. Histogram (continuous), possibly with a box plot to see outliers.

A3. Grouped bar chart (or stacked if the totals matter), built from a pivot_table with aggfunc='count'.

A4. Box plot per department.

A5. Scatter plot with hue = gender (relplot).

A6. Grouped box plot: x = Subject, y = Marks, hue = Trainer.

A7. IQR = 30. UF = 50 + 45 = 95; LF = 20 − 45 = −25. 100 > 95 → outlier.

🟢 11 · One-page cheat sheet

```
value_counts().reset_index()          counts table for a category
plt.bar / sns.barplot / plt.pie       category -> amounts or shares
plt.hist(x, bins=[...])               continuous -> distribution
plt.boxplot(x)  sns.boxplot(x=...)    continuous -> spread and outliers
UF = Q3 + 1.5*IQR   LF = Q1 - 1.5*IQR  fences
pivot_table(..., aggfunc='count').plot(kind='bar')   2 categories -> grouped bars
.plot.bar(stacked=True)               stacked bars
```

```
plt.boxplot([g1, g2])  sns.boxplot(x, y, data)       category + number
plt.scatter(x, y)  sns.relplot(..., hue=)            2 numbers (+ category)
```

21 October — Describing Data: Centre, Spread & Shape

Frequency tables · Mean, median, mode · Weighted mean · Range, IQR, variance, standard deviation · Skewness and kurtosis

On 21 October Sudeep Bhargava went through the three questions you ask of any column of numbers:

- WHERE is the middle? → mean, median, mode (measures of centre)
- HOW spread out is it? → range, IQR, variance, standard deviation (measures of spread)
- WHAT SHAPE is it? → skewness and kurtosis

Examples use the ERPData sheet (MaterialID) and the faithful sheet (eruptions). Every runnable cell uses small numbers you can verify by hand or simulated data with a fixed seed. The statistics note (Statistics & Model Evaluation) and the 5 September IQR note cover the same ideas in more depth; this note keeps to what was taught on the day.

How to read the grades: 🟢 start here · 🔵 core · 🟠 go deeper · 🔴 stretch

```
import numpy as np
import pandas as pd
import matplotlib
from matplotlib import pyplot as plt
import scipy
from scipy import stats
```

🟢 1 · Describing a categorical column: counts, proportions, percentages

For a categorical column the "summary" is a frequency table.

```
df = pd.read_excel('CDAC_DataBook.xlsx', sheet_name='ERPData')
np.unique(df.MaterialID)       # the distinct values, in ascending or alphabetical order
np.unique(df.MaterialID, return_counts=True)    # TWO arrays: the unique values, and the count of
each
```

```
x1 = np.unique(df.MaterialID, return_counts=True)
x1[1]                          # only the counts (second array)
prop = x1[1] / sum(x1[1])      # proportions (they add up to 1)
pct = prop * 100               # percentages (they add up to 100)
```

```
MaterialSummary = pd.DataFrame({'Name': np.unique(df.MaterialID),
                                'Counts': x1[1],
                                'Proportion': prop,
                                'Percentage': pct})
```

Cumulative totals — the running total up to each position — come from np.cumsum:

```
x2 = np.array([4, 3, 6, 2, 7])
np.cumsum(x2)      # [ 4  7 13 15 22 ]
```

▶ Try it — A frequency table by hand and with numpy

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(21)
material = rng.choice(["M101", "M102", "M103", "M104"], 80, p=[.15, .30, .25, .30])

x1 = np.unique(material, return_counts=True)
prop = x1[1] / sum(x1[1])
pct = prop * 100
MaterialSummary = pd.DataFrame({"Name": x1[0], "Counts": x1[1],
                                "Proportion": prop.round(3), "Percentage": pct.round(1)})
print(MaterialSummary)
print("\nproportions add up to", prop.sum().round(3))
print("\ncumsum of [4,3,6,2,7] =", np.cumsum(np.array([4, 3, 6, 2, 7])))
```

OUTPUT

```
   Name  Counts  Proportion  Percentage
0  M101      12       0.150        15.0
1  M102      27       0.338        33.8
2  M103      19       0.238        23.8
3  M104      22       0.275        27.5

proportions add up to 1.0

cumsum of [4,3,6,2,7] = [ 4  7 13 15 22]
```

🟢 2 · The mean

The mean is the balance point: add everything, divide by how many.

```
erupt = df.eruptions        # faithful sheet
sum(erupt) / len(erupt)
np.mean(erupt)              # the same answer
```

When is the mean the right choice? Two conditions were written on the board:

- the data should NOT have outliers
- the distribution should be symmetric

When either fails, the mean gets dragged and stops describing a "typical" value.

🟢 3 · The median

Sort the numbers; the median is the middle one. With an even count, it is the average of the two middle ones.

```
x2 = np.array([4, 3, 6, 2, 7])
np.median(x2)          # sorted: 2 3 4 6 7 -> middle = 4
np.median(erupt)
```

The median is not affected by outliers, which is why missing salaries are filled with the median (merge note). In a skewed or outlier-rich column, report the median.

▶ Try it — Mean vs median when there is an outlier

```
import numpy as np

salaries = np.array([30, 32, 35, 36, 38, 40])
print("without outlier -> mean = %.1f   median = %.1f" % (salaries.mean(),
np.median(salaries)))
salaries2 = np.append(salaries, 400)
print("with 400 added  -> mean = %.1f   median = %.1f" % (salaries2.mean(),
np.median(salaries2)))
print("\nThe mean jumped; the median barely moved.")
```

OUTPUT

```
without outlier -> mean = 35.2   median = 35.5
with 400 added  -> mean = 87.3   median = 36.0

The mean jumped; the median barely moved.
```

🔵 4 · The mode — and data with two modes

The mode is the value that occurs most often. NumPy has no mode function, so the class used Counter:

```
from collections import Counter
Counter(erupt).most_common()
# We have two modes - 1.867 and 4.5, each occurring 8 times
```

Two (or more) peaks = a bimodal (multimodal) column. The faithful geyser has short eruptions near 2 minutes and long ones near 4.5 minutes, so neither the mean nor the median describes either group — here the mode, or better, the histogram, tells the real story. Always look at the shape before trusting one number.

For a categorical column the mode is the only available centre: the most frequent label, such as the MaterialID with the largest count.

▶ Try it — Mode with Counter

```
from collections import Counter

values = [2.0, 4.5, 4.5, 1.9, 2.0, 4.5, 3.8, 2.0, 4.5, 2.0]
ranked = Counter(values).most_common()
print("most common first:", ranked)
top = ranked[0][1]
print("mode(s):", [v for v, c in ranked if c == top], "each appearing", top, "times")
```

OUTPUT

```
most common first: [(2.0, 4), (4.5, 4), (1.9, 1), (3.8, 1)]
mode(s): [2.0, 4.5] each appearing 4 times
```

🔵 5 · The weighted mean

When each value counts a different number of times (or has a different importance), weight it. The class example gave values with their frequencies:

```
values    = np.array([10, 20, 30, 40, 50])
frequency = np.array([ 7, 15, 35, 20, 10])
sum(values * frequency) / sum(frequency)
```

Work it out: 10×7 + 20×15 + 30×35 + 40×20 + 50×10 = 70 + 300 + 1050 + 800 + 500 = 2720. Total frequency = 87. Weighted mean = 2720 / 87 = 31.26.

The simple mean of 10, 20, 30, 40, 50 would be 30. The weighted mean is higher than 30 because 30, 40, 50 occur more often than 10 and 20.

▶ Try it — Weighted mean

```
import numpy as np

values = np.array([10, 20, 30, 40, 50])
frequency = np.array([7, 15, 35, 20, 10])
print("weighted mean :", sum(values * frequency) / sum(frequency))
print("np.average    :", np.average(values, weights=frequency))
print("simple mean   :", values.mean())
```

OUTPUT

```
weighted mean : 31.264367816091955
np.average    : 31.264367816091955
simple mean   : 30.0
```

🔵 6 · Measuring spread: range and IQR

Two columns can have the same mean and be completely different. Spread measures say how far the values wander from the centre.

Range

```
max(erupt) - min(erupt)
np.ptp(erupt)             # "peak to peak", the same thing
```

- Easy, but it uses only 2 values out of many.
- It is affected by outliers: one extreme value changes it completely.

Inter-quartile range (IQR)

```
np.percentile(erupt, 75) - np.percentile(erupt, 25)
```

- The width of the middle 50% of the data.
- It is (practically) NOT affected by outliers: the cell below shows the range jumping from 10 to 190 while the IQR moves only from 4.5 to 5.8.

▶ Try it — Range vs IQR with one outlier

```
import numpy as np

x = np.array([10, 12, 13, 15, 16, 18, 20])
y = np.append(x, 200)
for name, a in [("without outlier", x), ("with outlier 200", y)]:
    iqr = np.percentile(a, 75) - np.percentile(a, 25)
    print("%-18s range = %5.0f   IQR = %5.1f" % (name, np.ptp(a), iqr))
```

OUTPUT

```
without outlier    range =    10   IQR =   4.5
with outlier 200   range =   190   IQR =   5.8
```

🔵 7 · Variance and standard deviation — the formula first

Both use every value. The idea: for each value, find how far it is from the mean (its deviation), then average the squared deviations.

```
Step 1: mean = Σx / n
Step 2: deviation of each value = x − mean            (these ALWAYS add up to 0)
Step 3: squared deviation = (x − mean)²               (squaring removes the sign)
Step 4: sum of squared deviations = SSD
Step 5: population variance = SSD / n
        sample variance     = SSD / (n − 1)
Step 6: standard deviation = square root of the variance
```

In code, as written in class:

```
erupt_mean = np.mean(erupt)
df['deviation'] = df.eruptions - erupt_mean        # total of this column is always 0
df['sqd_deviation'] = (df.eruptions - erupt_mean)**2
ssd = sum(df.sqd_deviation)
ssd / df.shape[0]          # population variance   -> same as np.var(erupt)
ssd / (df.shape[0] - 1)    # sample variance       -> same as np.var(erupt, ddof=1)
(ssd / df.shape[0])**0.5          # population std deviation -> np.std(erupt)
(ssd / (df.shape[0] - 1))**0.5    # sample std deviation     -> np.std(erupt, ddof=1)
```

⚠ In the live chat one line was typed as ssd**0.5/(df.shape[0]-1). That divides the square root of SSD by (n−1), which is wrong. The correct form takes the square root of the WHOLE variance: (ssd/(n−1))**0.5, exactly as in the line above.

Using the library:

```
np.var(erupt, ddof=1)      # sample variance
np.std(erupt, ddof=1)      # sample standard deviation
```

- ddof=1 means "delta degrees of freedom = 1": divide by n − 1 instead of n. It gives the SAMPLE variance / SD, used when your data is a sample from a bigger population (the usual case).
- Without ddof (np.var(erupt)) NumPy divides by n: the POPULATION variance.
- Unit of variance = the SQUARE of the unit of the data (minutes²). Unit of standard deviation = the same as the data (minutes). That is why people quote the standard deviation.

▶ Try it — Variance and SD by hand, then by numpy

```
import numpy as np

x = np.array([4, 3, 6, 2, 7])
n = len(x)
mean = x.mean()
dev = x - mean
print("mean               :", mean)
print("deviations         :", dev.round(1), " sum =", abs(round(dev.sum(), 10)))
print("squared deviations :", (dev**2).round(2))
ssd = (dev**2).sum()
print("SSD                : %.2f" % ssd)
print("population variance: %.3f   np.var(x)         = %.3f" % (ssd/n, np.var(x)))
print("sample variance    : %.3f   np.var(x, ddof=1) = %.3f" % (ssd/(n-1), np.var(x,
ddof=1)))
print("sample SD          : %.4f  np.std(x, ddof=1) = %.4f" % ((ssd/(n-1))**0.5, np.std(x,
ddof=1)))
print("WRONG formula ssd**0.5/(n-1) gives: %.4f" % (ssd**0.5/(n-1)))
```

OUTPUT

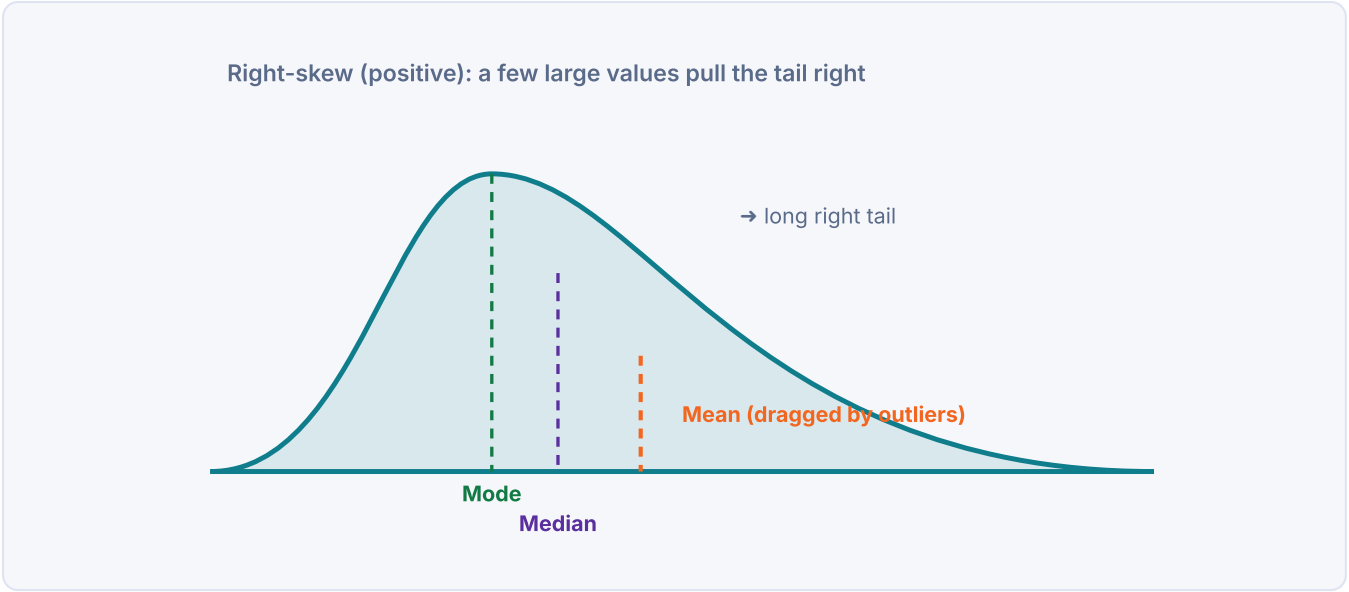
```
mean               : 4.4
deviations         : [-0.4 -1.4  1.6 -2.4  2.6]  sum = 0.0
squared deviations : [0.16 1.96 2.56 5.76 6.76]
SSD                : 17.20
population variance: 3.440   np.var(x)         = 3.440
sample variance    : 4.300   np.var(x, ddof=1) = 4.300
sample SD          : 2.0736  np.std(x, ddof=1) = 2.0736
WRONG formula ssd**0.5/(n-1) gives: 1.0368
```

🔵 8 · Shape 1: skewness (is the distribution lopsided?)

Skewness measures symmetry.

```
Peak of the distribution   Tail   Name                    Skewness   Example from class
on the LEFT                right  skewed to the right     > 0        salary distribution in a
company — many small salaries, few big ones
on the RIGHT               left   skewed to the left      < 0        age of people going to
Parliament — most are older, a few are young
in the middle, symmetric          symmetric               ≈ 0        heights of adults
```

The direction is named after the TAIL, not the peak. Salary: the peak is on the left (smaller values have a high frequency) and the long tail stretches right, so it is right-skewed with positive skewness. A quick check: in a right-skewed column the mean is pulled above the median.



The three histograms built in class with np.repeat (value, how many times):

```
x1 = np.repeat([1,2,3,4,5,6,7,8,9,10], [5,10,150,140,120,100,80,60,40,20])     # peak on the left
x2 = np.repeat([1,2,3,4,5,6,7,8,9,10], [20,40,60,80,100,120,140,150,10,5])     # peak on the
right
x3 = np.repeat([1,2,3,4,5,6,7,8,9,10,11], [2,5,10,20,30,40,30,20,10,5,2])      # symmetric
plt.hist(x1)
scipy.stats.skew(x1)
```

▶ Try it — Skewness of the three class histograms

```
import numpy as np
from scipy import stats

x1 = np.repeat([1,2,3,4,5,6,7,8,9,10], [5,10,150,140,120,100,80,60,40,20])
x2 = np.repeat([1,2,3,4,5,6,7,8,9,10], [20,40,60,80,100,120,140,150,10,5])
x3 = np.repeat([1,2,3,4,5,6,7,8,9,10,11], [2,5,10,20,30,40,30,20,10,5,2])

for name, x, text in [("x1", x1, "peak left  -> right skew"),
                      ("x2", x2, "peak right -> left skew"),
                      ("x3", x3, "symmetric")]:
    print("%s  skewness = %6.3f   mean = %.2f  median = %.1f   (%s)" %
          (name, stats.skew(x), x.mean(), np.median(x), text))
```

OUTPUT

```
x1  skewness =  0.489   mean = 5.29  median = 5.0   (peak left  -> right skew)
x2  skewness = -0.489   mean = 5.71  median = 6.0   (peak right -> left skew)
x3  skewness =  0.000   mean = 6.00  median = 6.0   (symmetric)
```

🔵 9 · Shape 2: kurtosis (how sharp is the peak?)

Kurtosis measures the sharpness of the peak compared with the normal (bell) curve. In scipy's convention a normal curve scores 0:

- kurtosis = 0 → the sharpness of the normal curve
- kurtosis positive → sharper peak than normal (and heavier tails)

- kurtosis negative → flatter than normal

The class examples:

```
x3 = np.repeat([1,2,3,4,5,6,7,8,9,10,11], [2,5,10,20,30,40,30,20,10,5,2])      # close to normal
scipy.stats.kurtosis(x3)
```

```
x1 = np.repeat([1,2,3,4,5,6,7,8,9,10], [5,10,15,20,400,20,15,10,5,2])          # very sharp peak
at 5
scipy.stats.kurtosis(x1)
```

```
x2 = np.repeat([1,2,3,4,5,6,7,8,9,10], [20,25,25,30,35,30,25,25,20,20])        # nearly flat
scipy.stats.kurtosis(x2)
```

(Note that the names x1, x2, x3 are reused in the class for different arrays. Use a new name each time in your own notebook, or re-run the cell that defines the array you need.)

▶ Try it — Kurtosis of the three shapes

```
import numpy as np
from scipy import stats

near_normal = np.repeat([1,2,3,4,5,6,7,8,9,10,11], [2,5,10,20,30,40,30,20,10,5,2])
sharp       = np.repeat([1,2,3,4,5,6,7,8,9,10], [5,10,15,20,400,20,15,10,5,2])
flat        = np.repeat([1,2,3,4,5,6,7,8,9,10], [20,25,25,30,35,30,25,25,20,20])

for name, x in [("near normal", near_normal), ("sharp peak", sharp), ("flat", flat)]:
    print("%-12s kurtosis = %6.3f" % (name, stats.kurtosis(x)))
print("\npositive = sharper than normal, negative = flatter than normal")
```

OUTPUT

```
near normal  kurtosis = -0.052
sharp peak   kurtosis =  6.850
flat         kurtosis = -1.023

positive = sharper than normal, negative = flatter than normal
```

🟠 10 · Putting it together: a describing-a-column routine

When you meet a new numeric column, answer in this order:

- 1. Plot it (histogram + box plot): one hump or two? symmetric? outliers?
- 2. Centre: mean if symmetric and clean; median if skewed or outliers; mode if categorical or multi-modal.
- 3. Spread: SD with the mean; IQR with the median.
- 4. Shape: skewness (direction), kurtosis (peakedness).

▶ Try it — One routine for a whole column

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(4)
```

```
col = np.append(rng.gamma(2.0, 20000, 300), [400000, 450000])   # salary-like, right skewed,
2 outliers

q1, q3 = np.percentile(col, [25, 75])
print("n        =", len(col))
print("mean     = %.0f   median = %.0f" % (col.mean(), np.median(col)))
print("SD       = %.0f   IQR    = %.0f" % (col.std(ddof=1), q3 - q1))
print("range    = %.0f" % np.ptp(col))
print("skewness = %.2f   kurtosis = %.2f" % (stats.skew(col), stats.kurtosis(col)))
print("mean > median and skew > 0  ->  right skewed: quote the MEDIAN and the IQR")
```

OUTPUT

```
n        = 302
mean     = 43232   median = 32299
SD       = 42642   IQR    = 40332
range    = 449784
skewness = 5.10   kurtosis = 41.70
mean > median and skew > 0  ->  right skewed: quote the MEDIAN and the IQR
```

🔵 11 · Practice questions

Q1. Data: 5, 8, 8, 9, 12, 14, 40. Find the mean, median and mode. Which would you quote?

Q2. Values 2, 4, 6 with frequencies 1, 3, 6. Weighted mean?

Q3. For x = 2, 4, 6, 8: find the population variance and the sample variance.

Q4. A column has skewness −0.9. Describe the shape and say where the peak is.

Q5. Kurtosis (scipy) of a column is −1.1. What does that tell you?

Q6. Why is the standard deviation preferred over the variance when reporting?

Q7. Which spread measure would you report for a column with outliers?

Answers

A1. Mean = 96/7 = 13.71; median = 9; mode = 8. Quote the median (9): 40 is an outlier that drags the mean.

A2. (2×1 + 4×3 + 6×6) / 10 = (2 + 12 + 36)/10 = 5.0.

A3. Mean = 5; deviations −3, −1, 1, 3; squares 9, 1, 1, 9; SSD = 20. Population variance = 20/4 = 5; sample variance = 20/3 = 6.67.

A4. Skewed to the left (negative skew): the peak is on the right and the long tail on the left.

A5. Flatter than a normal curve.

A6. It has the same unit as the data.

A7. The IQR (with the median).

🟢 12 · One-page cheat sheet

```
np.unique(x, return_counts=True)    frequency table
np.cumsum(x)                        running total
np.mean / np.median / Counter(x).most_common()    centre
np.average(values, weights=freq)    weighted mean
np.ptp(x)                           range
```

```
np.percentile(x,75) - np.percentile(x,25)         IQR
np.var(x, ddof=1)   np.std(x, ddof=1)             sample variance, SD
stats.skew(x)       > 0 right tail, < 0 left tail
stats.kurtosis(x)   > 0 sharper, < 0 flatter than normal
```

- Symmetric, no outliers → mean and SD.
- Skewed or outliers → median and IQR.

22 October — Probability & Bayes' Theorem

Probability from counts · AND / OR rules · Independent vs mutually exclusive · Conditional probability · Total probability · Bayes' theorem

On 22 October Sudeep Bhargava taught probability by working numbers out together: he put a question, students typed fractions into the chat (48/136, 38/48, 3/57 ...), and the class closed with a three-supplier problem solved with Bayes' theorem. The table of 136 cases was shown on screen and is not in the chat record, so this note re-teaches the same types of question on a table of its own (clearly marked), and works the supplier problem exactly as set in class.

How to read the grades: 🟢 start here · 🔵 core · 🟠 go deeper · 🔴 stretch

🟢 1 · What a probability is

Probability = (number of outcomes we want) ÷ (total number of equally likely outcomes). It is always between 0 and 1.

- 0 means impossible, 1 means certain.
- When you read a table of counts, the probability is simply a fraction of the counts: that is what the class typed as 48/136, 20/136 and 95/136 — "this many out of that many".
- The probability that an event does NOT happen is 1 − P(event). This is called the complement.

🔵 2 · Reading probabilities from a table (worked example of our own)

Practice table — 200 trainees, by batch and by result in the Python test. (Our own numbers, not from class.)

```
            Passed   Failed   Total
Batch 1       48       12       60
Batch 2       52       28       80
Batch 3       30       30       60
Total        130       70      200
```

The three kinds of probability, each read straight off the table:

- Marginal — one variable only, divide by the grand total 200.

```
P(Passed) = 130/200 = 0.65        P(Batch 2) = 80/200 = 0.40
```

- Joint — "A AND B": the single cell where both hold, divide by 200.

```
P(Batch 2 AND Passed) = 52/200 = 0.26
```

- Conditional — "A GIVEN B": restrict to the row (or column) of B and divide by ITS total.

```
P(Passed | Batch 2) = 52/80 = 0.65
P(Batch 1 | Passed) = 48/130 = 0.369
```

Compare the last two: the condition changes the denominator. This is the same move as in the chat: 8/136 is a joint probability (denominator = everyone), while 8/42 is a conditional one (denominator = only

the 42 people in the condition, a smaller group).

▶ Try it — Marginal, joint and conditional from a table

```
import pandas as pd

t = pd.DataFrame({"Passed": [48, 52, 30], "Failed": [12, 28, 30]},
                 index=["Batch 1", "Batch 2", "Batch 3"])
N = t.values.sum()
print(t, "\n")
print("P(Passed)              = %d/%d = %.3f" % (t.Passed.sum(), N, t.Passed.sum()/N))
print("P(Batch 2)             = %d/%d = %.3f" % (t.loc['Batch 2'].sum(), N, t.loc['Batch
2'].sum()/N))
print("P(Batch 2 AND Passed)  = %d/%d = %.3f" % (t.loc['Batch 2','Passed'], N, t.loc['Batch
2','Passed']/N))
print("P(Passed | Batch 2)    = %d/%d = %.3f" % (t.loc['Batch 2','Passed'], t.loc['Batch
2'].sum(),
                                                t.loc['Batch 2','Passed']/t.loc['Batch
2'].sum()))
print("P(Batch 1 | Passed)    = %d/%d = %.3f" % (t.loc['Batch 1','Passed'], t.Passed.sum(),
                                                t.loc['Batch 1','Passed']/t.Passed.sum()))
```

OUTPUT

```
         Passed  Failed
Batch 1      48      12
Batch 2      52      28
Batch 3      30      30

P(Passed)              = 130/200 = 0.650
P(Batch 2)             = 80/200 = 0.400
P(Batch 2 AND Passed)  = 52/200 = 0.260
P(Passed | Batch 2)    = 52/80 = 0.650
P(Batch 1 | Passed)    = 48/130 = 0.369
```

🔵 3 · The AND rule and the OR rule

OR (addition rule)

```
P(A or B) = P(A) + P(B) − P(A and B)
```

We subtract the overlap because people in both A and B were counted twice. From our table:

P(Batch 1 or Passed) = 60/200 + 130/200 − 48/200 = 0.30 + 0.65 − 0.24 = 0.71.

AND (multiplication rule)

```
P(A and B) = P(A) × P(B | A)
```

Read it as: first A happens, then B happens GIVEN that A has happened. Example: P(Batch 2 and Passed) = P(Batch 2) × P(Passed | Batch 2) = 0.40 × 0.65 = 0.26, which matches the joint probability in the table.

This is also how the first product was handled in the supplier problem: P(from A and defective) = 0.50 × 0.03 = 0.015.

🔵 4 · Independent or mutually exclusive? Two ideas that are often confused

The class was asked which word fits and students answered "independent" and "mutually exclusive". Here is the difference.

```
Mutually exclusive: A and B cannot happen together.       P(A and B) = 0
    Example: one product comes from supplier A OR from supplier B, never both.
Independent: knowing A tells you nothing about B.          P(B | A) = P(B), so P(A and B) = P(A)
× P(B)
    Example: two coins tossed separately.
```

Key facts:

- Mutually exclusive events are NOT independent (if A has happened, B certainly has not).
- The OR rule simplifies for mutually exclusive events: P(A or B) = P(A) + P(B), no overlap to subtract. That is why the three supplier probabilities could simply be added.
- Test for independence with the table: is P(Passed | Batch 2) equal to P(Passed)? Here 0.65 = 0.65, so Passed and Batch 2 are independent. For Batch 1, P(Passed | Batch 1) = 48/60 = 0.80 ≠ 0.65, so Batch 1 and Passed are NOT independent.

🔵 5 · The supplier problem — set in class

Three suppliers for a particular raw material: A, B and C.

- A supplies 50% of my requirements
- B supplies 30% of my requirements
- C supplies 20% of my requirements
- If the product comes from A, the probability of being defective is 3%
- If the product comes from B, the probability of being defective is 4%
- If the product comes from C, the probability of being defective is 5%

I find a defective product in my assembly. What is the probability that it came from A?

Think it through in two stages.

Stage 1 — "if I pick a product at random, how likely is it to be defective?" A defective product can reach me through any of the three suppliers, so add the three joint probabilities (they are mutually exclusive: a product has exactly one supplier).

```
P(A and D) = 0.50 × 0.03 = 0.015
P(B and D) = 0.30 × 0.04 = 0.012
P(C and D) = 0.20 × 0.05 = 0.010
P(D) = 0.015 + 0.012 + 0.010 = 0.037        (3.7% of all products are defective)
```

This is the LAW OF TOTAL PROBABILITY: P(D) = Σ P(supplier) × P(D | supplier).

Stage 2 — "given that THIS product is defective, where did it come from?" Take the share of the defects that comes from A:

```
P(A | D) = P(A and D) / P(D) = 0.015 / 0.037 = 0.405
```

The same for the others:

```
Supplier   Share of supply   P(D|supplier)   Joint P(and D)   Share of defects P(supplier | D)
A              0.50              0.03             0.015              0.405
```

```
B          0.30          0.04          0.012          0.324
C          0.20          0.05          0.010          0.270
Total      1.00                        0.037          1.000
```

Notice what the table tells a buyer: C supplies only 20% of the material but is the source of 27% of the defects, because its defect rate (5%) is the highest. A supplies half the material but only 40.5% of the defects. The posteriors always add up to 1.

(A slip seen in the class chat: the final division was typed as 0.015/0.37. The denominator is 0.037, not 0.37.)

▶ Try it — The supplier problem step by step

```
import pandas as pd

share = pd.Series({"A": 0.50, "B": 0.30, "C": 0.20})          # P(supplier)
defect = pd.Series({"A": 0.03, "B": 0.04, "C": 0.05})         # P(defective | supplier)

joint = share * defect                                        # P(supplier AND defective)
p_d = joint.sum()                                             # total probability of a
defective
posterior = joint / p_d                                       # P(supplier | defective)

out = pd.DataFrame({"share": share, "P(D|s)": defect, "joint": joint, "P(s|D)":
posterior.round(4)})
print(out)
print("\nP(defective) = %.3f" % p_d)
print("P(A | defective) = %.4f" % posterior["A"])
print("posteriors add up to", round(posterior.sum(), 6))
```

OUTPUT

```
   share  P(D|s)  joint  P(s|D)
A    0.5    0.03  0.015  0.4054
B    0.3    0.04  0.012  0.3243
C    0.2    0.05  0.010  0.2703

P(defective) = 0.037
P(A | defective) = 0.4054
posteriors add up to 1.0
```

🔵 6 · Bayes' theorem in one line

The supplier problem is Bayes' theorem. Written once, in symbols:

```
P(A | D) = P(D | A) × P(A) / P(D)
```

with P(D) found by total probability, P(D) = Σ P(D | Ai) × P(Ai).

- P(A) — PRIOR: what you believed before the evidence (A supplies 50%).
- P(D | A) — LIKELIHOOD: how likely the evidence is if A is the source (3%).
- P(A | D) — POSTERIOR: what you believe after the evidence (40.5%).

Warning: P(D | A) and P(A | D) are NOT the same thing. 3% of A's products are defective, but 40.5% of the defective products come from A. Mixing the two up is the most common probability mistake.

A second Bayes problem (our own example, to practise): a screening test. 1% of people have a disease. The test is positive for 95% of the people who have it (sensitivity) and also for 5% of healthy people (false positives). A person tests positive. How likely is it that they really have the disease?

```
P(disease and +)   = 0.01 × 0.95 = 0.0095
P(healthy and +)   = 0.99 × 0.05 = 0.0495
P(+)               = 0.0095 + 0.0495 = 0.0590
P(disease | +)     = 0.0095 / 0.0590 = 0.161  (about 16%)
```

Even with a "95% accurate" test, only about 1 positive in 6 is a true case, because healthy people are so much more numerous. This is exactly the precision idea in the Statistics & Model Evaluation note (precision = true positives ÷ all predicted positives).

▶ Try it — Screening-test Bayes problem, and a simulation to confirm the supplier answer

```
import numpy as np

# 1. the screening test
prior, sens, fpr = 0.01, 0.95, 0.05
p_pos = prior*sens + (1-prior)*fpr
print("P(disease | positive) = %.3f" % (prior*sens / p_pos))

# 2. confirm the supplier posterior by simulating 2,000,000 products
rng = np.random.default_rng(0)
n = 2000000
supplier = rng.choice(["A", "B", "C"], n, p=[.5, .3, .2])
p_def = np.where(supplier == "A", .03, np.where(supplier == "B", .04, .05))
defective = rng.random(n) < p_def
print("simulated share of defects      = %.4f" % defective.mean())
print("simulated P(A | defective)      = %.4f   (exact 0.4054)" % (supplier[defective] ==
"A").mean())
```

OUTPUT

```
P(disease | positive) = 0.161
simulated share of defects      = 0.0368
simulated P(A | defective)      = 0.4074   (exact 0.4054)
```

🟠 7 · Mistakes to avoid

- Dividing by the wrong total. Joint → grand total; conditional → total of the condition.
- Adding probabilities of events that overlap without subtracting the overlap.
- Multiplying probabilities of events that are not independent without using the conditional probability.
- Swapping P(D | A) and P(A | D).
- Forgetting the denominator in Bayes' theorem — it is the TOTAL probability of the evidence, not just one branch.
- Typing 0.37 for 0.037. Check that the denominator is larger than the numerator.

🔵 8 · Practice questions

Use the 200-trainee table of section 2.

Q1. P(Failed)? P(Batch 3 and Failed)?

Q2. P(Failed | Batch 3)? Are Failed and Batch 3 independent?

Q3. P(Batch 2 or Failed)?

Q4. Given that a trainee failed, what is the probability that they are from Batch 2?

Q5. A factory has two machines: M1 makes 60% of items with a 2% defect rate; M2 makes 40% with a 5% defect rate. What is the probability that an item is defective? If an item is defective, how likely is it from M2?

Q6. Two events are mutually exclusive with P(A) = 0.3 and P(B) = 0.5. Find P(A or B) and P(A and B).

Answers

A1. 70/200 = 0.35; 30/200 = 0.15.

A2. 30/60 = 0.50. P(Failed) = 0.35 ≠ 0.50, so NOT independent.

A3. P(Batch 2) = 0.40, P(Failed) = 0.35, P(Batch 2 and Failed) = 28/200 = 0.14 → 0.40 + 0.35 − 0.14 = 0.61.

A4. P(Batch 2 | Failed) = 28/70 = 0.40.

A5. P(D) = 0.6×0.02 + 0.4×0.05 = 0.012 + 0.020 = 0.032. P(M2 | D) = 0.020/0.032 = 0.625.

A6. P(A or B) = 0.8; P(A and B) = 0.

🟢 9 · One-page cheat sheet

```
P(A)            = favourable / total
P(not A)        = 1 − P(A)
P(A or B)       = P(A) + P(B) − P(A and B)
P(A and B)      = P(A) × P(B | A)
P(B | A)        = P(A and B) / P(A)
Mutually exclusive: P(A and B) = 0
Independent       : P(B | A) = P(B)  and  P(A and B) = P(A)P(B)
Total probability : P(D) = Σ P(Ai) P(D | Ai)
Bayes             : P(A | D) = P(D | A) P(A) / P(D)
```

- Joint → divide by the grand total. Conditional → divide by the condition's total.
- Prior → evidence → posterior.

23 October — Probability Distributions with scipy

Binomial · Poisson · Exponential · Normal · Student's t · pmf, cdf, sf, ppf

On 23 October Sudeep Bhargava moved from single probabilities to whole DISTRIBUTIONS: a distribution is the rule that tells you how likely every possible outcome is. The session used one set of imports and four scipy functions per distribution. These notes follow his examples (classes attended, challans issued, server breakdowns, Bangalore–Mumbai drive, LED bulbs) with the same numbers.

How to read the grades: 🟢 start here · 🔵 core · 🟠 go deeper · 🔴 stretch

The imports used all session:

```
import numpy as np
import pandas as pd
import matplotlib
from matplotlib import pyplot as plt
import seaborn as sns
import scipy
from scipy import stats
from scipy.stats import binom, poisson, norm, t, expon, f
```

🟢 1 · Which distribution? A decision map

```
Type of variable   Situation                                                  Distribution
DISCRETE           2 possible results; chance of ONE trial is known;          Binomial
(counts)           ask about a number of successes in n trials
                   counting events in a period; the average COUNT is known;   Poisson
                   the chance of one single event is unknown and tiny
CONTINUOUS         waiting time between two events (time between failures)     Exponential
(measurements)     bell-shaped measurement; mean and std deviation known      Normal
                   same, but sample is small (< 30) / std deviation estimated Student's t
```

Four functions work the same way for every distribution:

```
pmf(k, ...)   probability of EXACTLY k              (discrete distributions)
cdf(x, ...)   probability of x or LESS (cumulative) (all distributions)
sf(x, ...)    probability of MORE than x = 1 − cdf  (all distributions)
ppf(q, ...)   the value x below which a share q of the probability lies (inverse of cdf)
```

pmf = probability MASS function (discrete); cdf = cumulative distribution function; sf = survival function; ppf = percent point function.

🔵 2 · Binomial — successes out of n tries

Use it when: there are exactly two possible results on each try, the probability p of success on a single try is KNOWN from history, tries are independent, and you ask about a range of counts.

Class example: the probability that Sudeep comes to a class is 0.8. There are 15 classes.

```
binom.pmf(12, 15, 0.8)     # exactly 12 of 15 classes           -> 0.25
binom.cdf(12, 15, 0.8)     # at most (maximum) 12 of 15         -> 0.602
# minimum 12 of 15 means 12, 13, 14 or 15; the undesired numbers are 0 to 11
1 - binom.cdf(11, 15, 0.8)       # at least 12
```

```
binom.sf(11, 15, 0.8)           # the same thing in one call
binom.ppf(0.7, 15, 0.8)         # the number of classes up to which the cumulative probability
is at least 70%
```

The three words to translate:

- "exactly k" → pmf(k)
- "at most / maximum k" → cdf(k) (includes k itself)
- "at least / minimum k" → 1 − cdf(k − 1), or sf(k − 1)

▶ Try it — Binomial — attendance in 15 classes

```
from scipy.stats import binom

n, p = 15, 0.8
print("exactly 12      : %.5f" % binom.pmf(12, n, p))
print("at most 12      : %.5f" % binom.cdf(12, n, p))
print("at least 12     : %.5f  (1 - cdf(11))" % (1 - binom.cdf(11, n, p)))
print("at least 12 (sf): %.5f" % binom.sf(11, n, p))
print("ppf(0.7)        :", binom.ppf(0.7, n, p), " <- smallest k with cdf >= 0.7")
print("cdf(12) = %.3f < 0.7  and  cdf(13) = %.3f >= 0.7" % (binom.cdf(12, n, p),
binom.cdf(13, n, p)))
print("\nwhole distribution:")
for k in range(6, 16):
    print("  k=%2d  %-30s %.3f" % (k, "#" * int(binom.pmf(k, n, p) * 100), binom.pmf(k, n,
p)))
```

OUTPUT

```
exactly 12      : 0.25014
at most 12      : 0.60198
at least 12     : 0.64816  (1 - cdf(11))
at least 12 (sf): 0.64816
ppf(0.7)        : 13.0  <- smallest k with cdf >= 0.7
cdf(12) = 0.602 < 0.7  and  cdf(13) = 0.833 >= 0.7

whole distribution:
  k= 6                                0.001
  k= 7                                0.003
  k= 8  #                             0.014
  k= 9  ####                          0.043
  k=10  ##########                    0.103
  k=11  ##################            0.188
  k=12  ########################      0.250
  k=13  ######################        0.231
  k=14  #############                 0.132
  k=15  ###                           0.035
```

🔵 3 · Poisson — counting events in a fixed period

Use it when: you count how many times an event happens in a fixed stretch of time or space, the event is rare for any single moment (the chance of one single occurrence is unknown and very small), and you know the AVERAGE count from history. Only one number is needed: the average (mean) count.

Class example: the average number of challans issued per day is 40.

```
poisson.pmf(35, 40)       # exactly 35 challans today
poisson.cdf(35, 40)       # at most 35
```

```
1 - poisson.cdf(34, 40)   # at least 35
poisson.cdf(39, 40) - poisson.cdf(33, 40)     # between 34 and 39, both included
```

Why cdf(33) in the last line? "Between 34 and 39 inclusive" means the values 34, 35, ... 39. cdf(39) covers everything up to 39; we must remove everything up to 33, NOT up to 34, because 34 itself is wanted. This "subtract one from the lower end" step is the usual place to go wrong.

Binomial vs Poisson: binomial needs n and p (a fixed number of tries with a known chance); Poisson needs only the average count, for events with no natural "number of tries".

▶ Try it — Poisson — challans per day (average 40)

```
from scipy.stats import poisson

mu = 40
print("exactly 35        : %.4f" % poisson.pmf(35, mu))
print("at most 35        : %.4f" % poisson.cdf(35, mu))
print("at least 35       : %.4f" % (1 - poisson.cdf(34, mu)))
print("between 34 and 39 : %.4f" % (poisson.cdf(39, mu) - poisson.cdf(33, mu)))
check = sum(poisson.pmf(k, mu) for k in range(34, 40))
print("check by adding pmf(34..39): %.4f" % check)
```

OUTPUT

```
exactly 35        : 0.0485
at most 35        : 0.2424
at least 35       : 0.8061
between 34 and 39 : 0.3276
check by adding pmf(34..39): 0.3276
```

🔵 4 · Exponential — the gap between two events

Use it when: you model the TIME BETWEEN two successive events — the time between failures. The average gap is called MTBF (mean time between failures). Events that follow a Poisson count have exponential gaps.

Class example: in a year there are on average 70 server breakdowns.

- Mean time between breakdowns = 365 / 70 = 5.21 days.
- Question 1: if I am in Kharghar for 7 days, what is the probability that I will witness a breakdown?
- Question 2: how many days should I stay so that the probability of witnessing at least one breakdown is 99%?

IMPORTANT correction to the class code. The lines typed in the chat were

```
expon.cdf(7, 365/70)
expon.ppf(0.99, 365/70)
```

In scipy, the arguments after x are loc (a shift) and then scale. Written like this, 365/70 is taken as the LOC, the starting point of the distribution, not the mean. The mean of the exponential distribution goes into scale. The correct calls are:

```
expon.cdf(7, scale=365/70)       # 0.739  -> a 74% chance of seeing a breakdown in 7 days
expon.ppf(0.99, scale=365/70)    # 24.0   -> stay about 24 days for a 99% chance
```

With the original lines the answers come out as 0.832 and 9.82 days, which are wrong. A quick sanity check that catches this: with 70 breakdowns a year (one every 5 days), staying only 9.8 days could not give 99% certainty.

Why: the exponential cdf is 1 − exp(−x / mean). With mean = 5.214 and x = 7: 1 − exp(−1.342) = 1 − 0.261 = 0.739. For the ppf: x = −mean × ln(1 − 0.99) = 5.214 × 4.605 = 24.01 days.

▶ Try it — Exponential — the loc/scale trap, and the check by formula

```
import numpy as np
from scipy.stats import expon

mtbf = 365 / 70
print("mean time between breakdowns = %.3f days" % mtbf)
print("\nCORRECT (scale = mean):")
print("  P(breakdown within 7 days) = %.4f" % expon.cdf(7, scale=mtbf))
print("  days for a 99%% chance     = %.2f" % expon.ppf(0.99, scale=mtbf))
print("\nformula check: 1 - exp(-7/mtbf) = %.4f ;  -mtbf*ln(0.01) = %.2f" %
      (1 - np.exp(-7/mtbf), -mtbf*np.log(0.01)))
print("\nWRONG (mean passed as loc):")
print("  expon.cdf(7, 365/70)       = %.4f" % expon.cdf(7, 365/70))
print("  expon.ppf(0.99, 365/70)    = %.2f" % expon.ppf(0.99, 365/70))
```

OUTPUT

```
mean time between breakdowns = 5.214 days

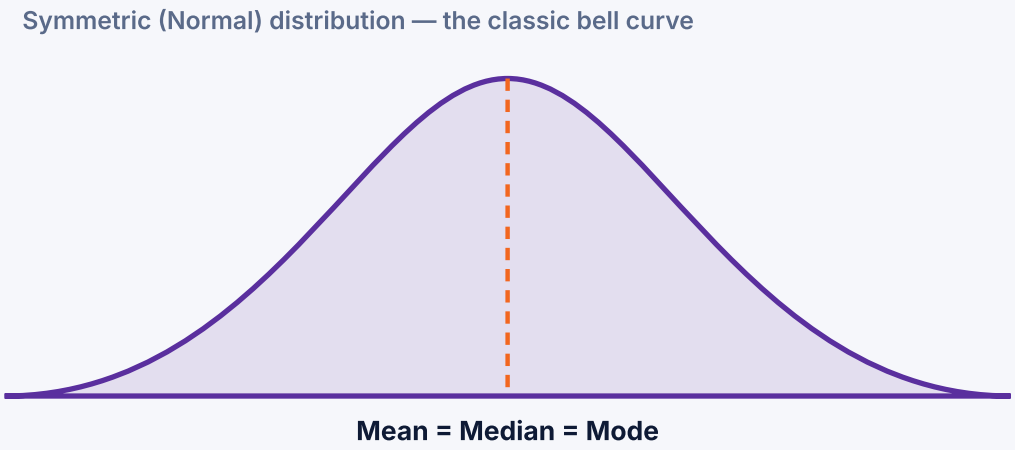
CORRECT (scale = mean):
  P(breakdown within 7 days) = 0.7388
  days for a 99% chance     = 24.01

formula check: 1 - exp(-7/mtbf) = 0.7388 ;  -mtbf*ln(0.01) = 24.01

WRONG (mean passed as loc):
  expon.cdf(7, 365/70)       = 0.8323
  expon.ppf(0.99, 365/70)    = 9.82
```

🔵 5 · Normal — the bell curve

Use it when: the variable is a continuous measurement with a symmetric bell shape, and you know the mean and the standard deviation. TWO parameters define a normal curve: the mean (where the centre is) and the standard deviation (how wide).



Class example: the drive time from Bangalore to Mumbai has a mean of 15 hours and a standard deviation of 2 hours.

```
norm.cdf(14, 15, 2)                          # P(less than 14 hours)
1 - norm.cdf(14, 15, 2)                      # P(more than 14 hours)
norm.sf(14, 15, 2)                           # the same
norm.cdf(16.2, 15, 2) - norm.cdf(13.5, 15, 2) # P(between 13.5 and 16.2 hours)
```

The order of the arguments is (value, mean, std deviation). For a continuous variable, "less than" and "at most" are the same, because the chance of hitting one exact value is zero.

The standard normal curve and the z-score

The standard normal curve has mean 0 and standard deviation 1. Any value x from a normal curve can be moved onto it with

```
z = (x − mean) / std deviation
```

```
z_14 = (14 - 15) / 2          # = -0.5: 14 hours is half a standard deviation below the mean
norm.cdf(z_14)                # same answer as norm.cdf(14, 15, 2)
norm.cdf(z_14, 0, 1)
```

z-scores allow all normal problems to use one table and let you compare values from different scales: a z of −0.5 means "half a standard deviation below average" whatever the unit.

The reverse question — ppf. LED bulbs: lifetime follows a normal distribution with mean 500 hours and standard deviation 150 hours. What guaranteed life can the company give so that 90% of bulbs last longer than it?

```
norm.ppf(1 - 0.9, 500, 150)     # = norm.ppf(0.1, 500, 150) = 307.8 hours
```

Think of the picture: 90% of bulbs must lie ABOVE the guarantee, so only 10% lie below it. The guarantee is the 10th percentile. A student first typed 1 − norm.ppf(0.9, 500, 150); that gives about −691, an impossible lifetime — you cannot subtract the ppf from 1, because the ppf returns hours, not a probability. Convert the probability first (1 − 0.9), then take the ppf.

▶ Try it — Normal — drive time and LED guarantee

```
from scipy.stats import norm

mean, sd = 15, 2
print("P(< 14 h)           = %.4f" % norm.cdf(14, mean, sd))
print("P(> 14 h)           = %.4f   (sf: %.4f)" % (1 - norm.cdf(14, mean, sd), norm.sf(14,
mean, sd)))
print("P(13.5 < T < 16.2)  = %.4f" % (norm.cdf(16.2, mean, sd) - norm.cdf(13.5, mean, sd)))

z14 = (14 - mean) / sd
print("\nz for 14 hours      = %.2f ;  norm.cdf(z) = %.4f" % (z14, norm.cdf(z14)))

print("\nLED bulbs, mean 500, sd 150:")
print("  guarantee (90%% last longer) = %.1f hours" % norm.ppf(1 - 0.9, 500, 150))
print("  WRONG 1 - ppf(0.9)          = %.1f hours  <- impossible" % (1 - norm.ppf(0.9, 500,
150)))
print("  check: P(life > guarantee)  = %.3f" % norm.sf(norm.ppf(0.1, 500, 150), 500, 150))
```

OUTPUT

```
P(< 14 h)           = 0.3085
P(> 14 h)           = 0.6915   (sf: 0.6915)
P(13.5 < T < 16.2)  = 0.4991

z for 14 hours      = -0.50 ;  norm.cdf(z) = 0.3085

LED bulbs, mean 500, sd 150:
  guarantee (90% last longer) = 307.8 hours
  WRONG 1 - ppf(0.9)          = -691.2 hours  <- impossible
  check: P(life > guarantee)  = 0.900
```

🔵 6 · Student's t — when the normal curve is not safe

The normal distribution may be used only if

- the sample size is 30 or more, or you are working with population data, AND
- the population standard deviation is known.

When the sample is small (below 30) and the standard deviation comes from that small sample, use the t distribution. It is bell-shaped like the normal but has fatter tails, which honestly reflects the extra uncertainty. It has one parameter, the degrees of freedom (df) = n − 1; as df grows it becomes the normal curve.

Class example: the same drive, but the mean 15 and standard deviation 2 now come from a SAMPLE OF 20 TRIPS, so df = 19. Because 20 < 30, the normal cannot be used.

```
z_14 = (14 - 15) / 2
t.cdf(z_14, 19)                # P(less than 14 hours)
1 - t.cdf(z_14, 19)            # P(more than 14 hours)
z_135 = (13.5 - 15) / 2
z_162 = (16.2 - 15) / 2
t.cdf(z_162, 19) - t.cdf(z_135, 19)       # P(between 13.5 and 16.2 hours)
```

The steps are always: (1) convert to a standardised value, (2) use t.cdf with df = n − 1 instead of norm.cdf.

🟠 Go deeper. The t answers are close to the normal answers (about 0.311 vs 0.309 below 14 hours) and differ most in the tails. Strictly, the t distribution describes the sample MEAN; for a single new trip the

textbook treatment adds a small correction factor. At this stage of the course use the method as taught: small sample → t with n − 1 degrees of freedom.

▶ Try it — Normal vs t for the same question

```
from scipy.stats import norm, t

mean, sd, n = 15, 2, 20
df = n - 1
z14 = (14 - mean) / sd
z135 = (13.5 - mean) / sd
z162 = (16.2 - mean) / sd

print("                        normal    t (df=19)")
print("P(< 14 h)           %8.4f  %9.4f" % (norm.cdf(z14), t.cdf(z14, df)))
print("P(> 14 h)           %8.4f  %9.4f" % (norm.sf(z14), t.sf(z14, df)))
print("P(13.5 < T < 16.2)  %8.4f  %9.4f" % (norm.cdf(z162) - norm.cdf(z135),
                                            t.cdf(z162, df) - t.cdf(z135, df)))
print("\nP(|z| > 3):  normal = %.4f   t(df=19) = %.4f   <- fatter tails" % (2*norm.sf(3),
2*t.sf(3, df)))
```

OUTPUT

```
                       normal    t (df=19)
P(< 14 h)             0.3085     0.3114
P(> 14 h)             0.6915     0.6886
P(13.5 < T < 16.2)    0.4991     0.4910

P(|z| > 3):  normal = 0.0027   t(df=19) = 0.0074   <- fatter tails
```

🟠 7 · Mistakes to avoid

- "At least k" computed as 1 − cdf(k). It must be 1 − cdf(k − 1) for a discrete variable (or sf(k − 1)).
- "Between a and b inclusive" for a discrete variable: cdf(b) − cdf(a − 1).
- Passing the mean of an exponential as the second positional argument. Always write scale=mean.
- 1 − ppf(...) instead of ppf(1 − probability).
- Using the normal curve with a small sample and an estimated standard deviation; use t.
- Mixing up pmf (exact, discrete only) and cdf (up to and including).
- Forgetting that the order for norm is (value, mean, sd).

🔵 8 · Practice questions

Q1. A call centre agent resolves a call with probability 0.7. Of 10 calls, what is the probability that exactly 7 are resolved? At least 9?

Q2. A website gets on average 12 errors per day. P(exactly 10 errors)? P(more than 15)?

Q3. A machine fails on average once every 40 hours (exponential). Probability that it runs at least 60 hours without failing?

Q4. Delivery time is normal with mean 30 min and SD 5 min. P(delivery within 25 min)? Within what time are 95% of deliveries completed?

Q5. For Q4, convert 38 minutes to a z-score and say what it means.

Q6. A sample of 12 deliveries has mean 30 and SD 5. Which distribution should you use to find P(time < 25)? Write the code.

Answers (check with the cells above, then run your own)

A1. binom.pmf(7, 10, 0.7) = 0.2668; at least 9 = binom.sf(8, 10, 0.7) = 0.1493.

A2. poisson.pmf(10, 12) = 0.1048; more than 15 = poisson.sf(15, 12) = 0.1556.

A3. Survival at 60 hours = expon.sf(60, scale=40) = exp(−1.5) = 0.2231.

A4. norm.cdf(25, 30, 5) = 0.1587; norm.ppf(0.95, 30, 5) = 38.2 minutes.

A5. z = (38 − 30)/5 = 1.6: 1.6 standard deviations above the average delivery time.

A6. n = 12 < 30 and the SD is estimated from the sample → t with df = 11: z = (25 − 30)/5 = −1; t.cdf(-1, 11) = 0.169.

🟢 9 · One-page cheat sheet

```
binom.pmf(k, n, p)       exactly k of n          |  at most: binom.cdf(k, n, p)
poisson.pmf(k, mu)       exactly k events        |  at most: poisson.cdf(k, mu)
expon.cdf(x, scale=mean_gap)                     time-gap problems: ALWAYS scale=
norm.cdf(x, mean, sd)    P(X < x)                |  norm.sf(x, mean, sd) P(X > x)
norm.ppf(q, mean, sd)    value with share q below it
t.cdf(z, n-1)            small sample: z = (x - mean)/sd
```

```
at least k   ->  1 - cdf(k - 1)   or   sf(k - 1)   (discrete)
between a..b ->  cdf(b) - cdf(a - 1)               (discrete, inclusive)
between a..b ->  cdf(b) - cdf(a)                   (continuous)
```

- Discrete count with known p and n → binomial. Count with known average rate → Poisson.
- Gap between events → exponential. Measurement, big sample → normal. Small sample → t.